

Embedded Linux System

**นายเกียร ฤกุล
นายสถาพร สงวนวงษ์**

**ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
ภาควิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2547**

Embedded Linux System

โดย

นายเกษร มฤกุล

นายสถาพร สงวนวงษ์

อาจารย์ที่ปรึกษา

รศ. ดร. บุญธีร์ เครือตราฐ

ผศ. กฤตวัน ศิริบูรณ์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2547

ปริญญานิพนธ์ ปีการศึกษา 2547

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เรื่อง Embedded Linux System

คณะผู้จัดทำ นายเกียร มฤกุล รหัส 45015355

นายสถาพร สงวนวงษ์ รหัส 45015385

.....อาจารย์ที่ปรึกษา
(รศ. ดร. บุญธีร์ เกียรติราชู)

.....อาจารย์ที่ปรึกษา
(ผศ. กฤตวัน ศิริบุญรัตน์)

Embedded Linux System

นายเกษร มฤกุล 45015355
 นายสถาพร สงวนวงษ์ 45015385
 รศ. ดร. บุญธีร์ เครือตราฐ อาจารย์ที่ปรึกษา
 ผศ. กฤตวัน ศิริบุญณ์ อาจารย์ที่ปรึกษา
 ปีการศึกษา 2547

บทคัดย่อ

ระบบสมองกลฝังตัว (Embedded System) เป็นระบบคอมพิวเตอร์ที่ถูกใช้งานอย่างกว้างขวาง และเข้ามาบทบาทในชีวิตประจำวันของบุคคลทั่วไปมากขึ้น ตั้งแต่นาฬิกาข้อมือ โทรศัพท์มือถือ ไปจนถึง ส่วนควบคุมขนาดเล็กในโรงงานอุตสาหกรรมล้วนแล้วเป็นผลมาจากประติสัมพันธ์ระบบสมองกลฝังตัว ทั้งสิ้น ระบบปฏิบัติการลินุกซ์เป็นอีกทางเลือกหนึ่งในการพัฒนาระบบสมองกลฝังตัว ด้วยเหตุที่ลินุกซ์มีความสามารถในการปรับเปลี่ยนตัวเองให้เหมาะสมกับอุปกรณ์ได้หลากหลายตระกูล และได้รับความนิยมจากนักพัฒนาในปัจจุบัน ด้วยเหตุนี้จึงเกิดระบบสมองกลฝังตัวที่ใช้ลินุกซ์เป็นระบบปฏิบัติการเรียกว่า Embedded Linux Systems

ผู้จัดทำได้ทำการศึกษาการทำงานของลินุกซ์และระบบสมองกลฝังตัว เพื่อนำมาใช้สร้างระบบจำลองการควบคุมเครื่องที่สามารถเคลื่อนที่ได้ทั้งในแนวดิ่งและแนวนอน เพื่อทดสอบความสามารถและประสิทธิภาพของระบบสมองกลฝังตัวที่ทำงานบนลินุกซ์ ซึ่งผู้จัดทำได้นำ Debian Linux มาเป็นระบบปฏิบัติการหลักในการทำงาน ด้วยเหตุที่มีความกระชับและง่ายต่อการปรับแต่ง นอกจากนี้ผู้จัดทำได้เขียนโปรแกรมในลักษณะ Linux Device Driver Module ซึ่งเป็นโปรแกรมที่สามารถผนวกการทำงานเข้ากับ Linux Kernel ได้เป็นอย่างดี

ปัญญานิพนธ์นี้เป็นเพียงตัวอย่างการใช้งานระบบสมองกลฝังตัวที่ใช้ลินุกซ์ โดยแนวทางการพัฒนาอาจนำไปพัฒนาใช้งานในระบบที่ใหญ่ขึ้น เช่น ระบบจัดเก็บสินค้าที่มีขนาดใหญ่ที่มีการใช้คอมพิวเตอร์ช่วยในการจัดเก็บเพื่อความแม่นยำ หรือระบบขายของอัตโนมัติที่สามารถเก็บประวัติการซื้อขายของลูกค้าแต่ละคนเพิ่มเติมจากระบบเดิม เป็นต้น ดังนั้นการพัฒนาระบบสมองกลฝังตัวด้วยลินุกซ์จึงเป็น การศึกษาที่มีเป็นประโยชน์ต่อโปรแกรมเมอร์, นักพัฒนาอุปกรณ์คอมพิวเตอร์และอิเล็กทรอนิกส์ ในอนาคต

Embedded Linux System

Mr.Kaeyoon Maruekun 45015355

Mr.Sthaporn Sa-ngounwong 45015385

Assoc.Prof.Dr.Boontee Krutrathue Advisor

Asst.Prof.Kritawan Siriboon Advisor

Academic Year 2004

ABSTRACT

An embedded system is a computer system which is world wide used and involved in our daily life. Lot of things from watch, mobile phone, and so on control system in small factory are made from it. Linux System is one choice for developing the embedded system. The Linux has the ability in adjustment with many equipments and well known by many developers. The embedded system, developed with Linux, called Embedded Linux System.

A team studies Linux and Embedded System. The study developed the system to control crane in vertical and horizontal direction. Debian Linux, which is compact and easy adjustment, is selected as main system for testing ability and efficiency of Embedded Linux System. Furthermore the team developed the computer's program in Linux Device Driver Module which can well operate with Linux Kernel.

This project is a sample for developing Linux in Embedded System. This development can be used in larger system. There are many examples. A large inventory is computerized for higher accuracy. An automated trade system can be stored and added sale history from existing system. Therefore the development of the Embedded Linux System is the worth for programmers and other computer and electronic developers.

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้สำเร็จได้ด้วยความกรุณาจากอาจารย์ที่ปรึกษา รศ.ดร.บุญธีร์ เครือตราช และ ผศ.กฤตวัน ศิริบูรณ์ ที่ให้ความช่วยเหลือ ให้คำชี้แนะ ช่วยแก้ไขปัญหา ตลอดจนให้ความรู้และประสบการณ์ที่ดีแก่ผู้จัดทำ

ขอบคุณเพื่อนๆ นักศึกษาทุกคนสำหรับกำลังใจที่มีให้และความช่วยเหลือทุกๆ อย่างซึ่งมากมายเหลือเกิน

ขอขอบพระคุณบิดามารดา และญาติพี่น้องทุกคนที่เป็นกำลังใจ ให้คำแนะนำเป็นอย่างดีและช่วยให้ปริญญานิพนธ์นี้ประสบผลสำเร็จลุล่วงไปได้ด้วยดี

ปริญญานิพนธ์นี้คงไม่สามารถสำเร็จลุล่วงได้ หากไม่ได้รับการสนับสนุนจากบุคคลดังรายนามข้างต้น ผู้จัดทำจึงใคร่ขอขอบพระคุณทุกท่านดังที่กล่าวมาเป็นอย่างสูง มา ณ โอกาสนี้

สำหรับคุณงานความดีอันใดที่เกิดจากปริญญานิพนธ์นี้ ผู้จัดทำขอมอบให้กับบิดามารดาซึ่งเป็นที่รักและเคารพยิ่ง ตลอดจนครูอาจารย์ที่เคารพทุกท่านที่ได้ประสาทวิชาความรู้และถ่ายทอดประสบการณ์ที่ดีให้แก่ผู้จัดทำ

เกษร มฤกุล

สถาพร สงวนวงษ์

สารบัญ

หน้า

บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ	III
สารบัญ	IV
สารบัญภาพ	VII
บทที่ 1 บทนำ	1
1.1 ความสำคัญและที่มา	1
1.2 วัตถุประสงค์	2
1.3 ประโยชน์ที่คาดว่าจะได้รับ	2
1.4 ทฤษฎีและแนวความคิดที่ใช้ในการทำงาน	2
1.5 ขอบเขตของงานวิจัย	2
1.6 วิธีการดำเนินงาน	3
บทที่ 2	5
2.1 ระบบที่ทำงานด้วยไมโครคอนโทรลเลอร์ (Microcontrollers)	5
2.2 ระบบที่ทำงานด้วยไมโครโพรเซสเซอร์ (Microprocessors)	5
2.3 ระบบขนาดใหญ่ (Large-scale systems)	5
2.3.1 Unix Operating System	6
2.3.2 Unix Clone Operating System	6
2.3.3 Windows Operating System	6
บทที่ 3	7
3.1 บทนำ	7
3.2 การศึกษาข้อมูล	7
3.2.1 ศึกษาการทำงานของ Linux kernel	7
3.2.2 ศึกษาวิธีการเขียน Linux Device driver	8
3.3 การออกแบบระบบ	9
3.3.1 โครงสร้างและการทำงานของระบบโดยรวม	9
3.3.2 การออกแบบโปรแกรม	10
3.3.2.1 คุณสมบัติของโปรแกรม	10
3.3.2.2 โครงสร้างของโปรแกรม	10
3.3.2.3 จำนวนและขนาดของข้อมูลที่ใช้งาน	12
3.3.2.4 State diagram	13
3.3.2.5 Sequence diagram	14

3.3.2.6 Flowchart	15
3.3.2.7 Crane_move procedure	16
3.3.2.8 Interrupt_action procedure	17
3.3.2.9 Move to 0,0 position	18
3.3.3 การเลือก Embedded Linux server	18
3.3.3.1 Hardware Specifications	19
3.3.3.2 Software Specifications	19
3.3.4 การออกแบบวงจรเครนอินเตอร์เฟส	19
3.3.4.1 คุณสมบัติของเครนอินเตอร์เฟส	20
3.3.4.2 crane interface block diagram	20
3.3.4.3 การออกแบบวงจร	20
3.3.5 การออกแบบเครนและ โครง	22
3.3.5.1 คุณสมบัติของเครนและ โครง	22
3.3.5.2 การออกแบบโครงสร้างของเครน	23
3.3.5.3 การออกแบบโครงสร้างของ โครง	23
3.4 การติดตั้งและปรับแต่งเซิร์ฟเวอร์	24
3.4.1 การติดตั้ง Debian Linux	24
3.4.2 การปรับแต่งเซิร์ฟเวอร์	24
3.4.2.1 การตรวจสอบเวอร์ชันของ linux kernel และ linux include file	25
3.4.2.2 การตรวจสอบการทำงานของเน็ตเวิร์กการ์ด	25
3.4.2.3 การตรวจสอบการทำงานของเทลเน็ตและเอฟทีพี	26
3.4.2.4 การตรวจสอบการทำงานของเว็บเซิร์ฟเวอร์	27
3.5 การเขียนโปรแกรม	28
3.5.1 โปรแกรมส่วน Crane controller	28
3.5.1.1 โปรแกรมส่วน Global declaration	28
3.5.1.2 โปรแกรมส่วน Module initialize	30
3.5.1.3 โปรแกรมส่วนควบคุมสตีปปีงมอเตอร์	32
3.5.1.4 โปรแกรมส่วนการทำงานเมื่อเกิดอินเตอร์รัพ	35
3.5.1.5 โปรแกรมส่วนแสดงข้อมูลให้กับผู้ใช้	36
3.5.2 โปรแกรมติดต่อกับผู้ใช้ผ่านเว็บเพจ	37
3.5.3 ทดสอบการทำงาน	40
3.6 การจัดทำเครนอินเตอร์เฟส	40
3.6.1 การเตรียมอุปกรณ์	40
3.6.2 การประกอบวงจรเครนอินเตอร์เฟส	41

3.6.3 ทดสอบการทำงาน	42
3.7 การจัดทำเครนและโครง	42
3.7.1 การเตรียมอุปกรณ์	42
3.7.2 การประกอบอุปกรณ์	42
3.7.2.1 ส่วนโครงอะลูมิเนียม	42
3.7.2.2 ส่วนติดตั้งสแต็ปป์มอเตอร์	42
3.7.2.3 การติดตั้งลิมิตสวิทช์	43
3.7.2.4 โพล์รับสายพาน	44
3.7.2.5 ส่วนปรับความตึงสายพาน	44
3.7.2.6 การติดตั้งอุปกรณ์เซ็นเซอร์	45
3.7.2.7 ส่วนประกอบอื่นๆ	45
3.7.3 ทดสอบการทำงาน	47
บทที่ 4 ผลการทำงานและทดสอบระบบ	48
4.1 ผลการทดสอบระบบ	48
4.2 ปัญหาที่พบ	50
4.2.1 ปัญหาเกี่ยวกับเซิร์ฟเวอร์	50
4.2.2 ปัญหาเกี่ยวกับโปรแกรม	50
4.2.3 ปัญหาเกี่ยวกับเครนอินเตอร์เฟส	50
4.2.4 ปัญหาเกี่ยวกับเครนและโครง	50
บทที่ 5 สรุป	52
5.1 สรุปผลการทำงาน	52
5.2 ข้อเสนอแนะ	52
ภาคผนวก	
ภาคผนวก ก โปรแกรมที่สมบูรณ์ของ Crane Controller	53
ภาคผนวก ข โปรแกรมที่สมบูรณ์ของ main.cgi	71
ภาคผนวก ค วงจรเซ็นเซอร์ตรวจจับเส้นของบริษัท ETT	75
ภาคผนวก ง คุณสมบัติของไอซี ULN2003	76
ภาคผนวก จ วงจรที่สมบูรณ์ของ Crane interface card	86
ภาคผนวก ฉ คุณสมบัติของสแต็ปป์มอเตอร์ PM55L	88
บรรณานุกรม	89

สารบัญภาพ

	หน้า
รูปที่ 3-1 แสดงการทำงานของคำสั่ง insmod และ rmmod	8
รูปที่ 3-2 แสดงโครงสร้างโดยรวมของระบบ	9
รูปที่ 3-3 แสดงลักษณะโครงสร้างของ Crane controller device driver	10
รูปที่ 3-4 แสดง state diagram ของระบบ	13
รูปที่ 3-5 แสดง sequence diagram ของระบบ	14
รูปที่ 3-6 แสดงขั้นตอนการทำงานของฟังก์ชัน init_module()	15
รูปที่ 3-7 แสดงขั้นตอนการทำงานเมื่อมีการสั่งให้เครนเคลื่อนที่	16
รูปที่ 3-8 แสดงขั้นตอนการทำงานเมื่อมีอินเตอร์รัพ	17
รูปที่ 3-9 แสดงขั้นตอนการทำงานของ Move_to_00 ()	18
รูปที่ 3-10 แสดงเครื่องคอมพิวเตอร์ที่ทำหน้าที่ Embedded Linux Server	19
รูปที่ 3-11 แสดง Block diagram ของวงจรเครนอินเตอร์เฟส	20
รูปที่ 3-12 แสดงวงจรรับสัญญาณ dry contact จากอุปกรณ์ภายนอก	21
รูปที่ 3-13 แสดงวงจรรับสัญญาณจากอุปกรณ์เซ็นเซอร์ภายนอก	21
รูปที่ 3-14 แสดงวงจรการปรับระดับสัญญาณที่ส่งไปยังสแต็ปมอเตอร์	22
รูปที่ 3-15 แสดงวงจรเพาเวอร์ซัพพลายเพื่อจ่ายไฟให้กับสแต็ปมอเตอร์	22
รูปที่ 3-16 แสดงลักษณะโครงสร้างของเครน	23
รูปที่ 3-17 แสดงลักษณะโครงสร้างของโครง	24
รูปที่ 3-18 แสดงแผ่นวงจรเครนอินเตอร์เฟสหลังจากประกอบอุปกรณ์เรียบร้อยแล้ว	41
รูปที่ 3-19 แสดงวงจรเพาเวอร์ซัพพลาย	41
รูปที่ 3-20 แสดงการติดตั้งสแต็ปมอเตอร์ควบคุมแวนอน	43
รูปที่ 3-21 แสดงการติดตั้งสแต็ปมอเตอร์ควบคุมแวนตั้ง	43
รูปที่ 3-22 แสดงการติดตั้งลิมิตสวิตช์เข้ากับส่วนเคลื่อนที่แวนอน	44
รูปที่ 3-23 แสดงการติดตั้งโพเลียดสายพานแวนอน	44
รูปที่ 3-24 แสดงการใช้เคเบิลไทร์ในการปรับความตึงของสายพาน	45
รูปที่ 3-25 แสดงการติดตั้งเซ็นเซอร์สำหรับตรวจจับการเคลื่อนที่	45
รูปที่ 3-26 แสดงเครนและโครงที่สมบูรณ์	46
รูปที่ 4-1 แสดงส่วนติดต่อกับผู้ใช้เมื่อระบบอยู่ในตำแหน่งเริ่มต้น	48
รูปที่ 4-2 แสดงส่วนติดต่อกับผู้ใช้เมื่อผู้ใช้กรอกตำแหน่งที่ต้องการให้เครนเคลื่อนที่	49
รูปที่ 4-3 แสดงส่วนติดต่อกับผู้ใช้เมื่อเครนอยู่ในตำแหน่งเป้าหมาย	49

บทที่ 1

บทนำ

1.1 ความสำคัญและที่มา

ระบบสมองกลฝังตัว (Embedded systems หรือ Realtime system) มีจุดเริ่มต้นมาจากการต้องการเครื่องมือที่สามารถดูแลตัวเอง สามารถจัดการปัญหาต่างๆด้วยตัวเอง และระบบสามารถตอบสนองต่อสิ่งที่เกิดขึ้นได้ทันที โดยมีมนุษย์เป็นเพียงผู้สั่งให้เริ่มทำงานเท่านั้น ในการใช้งานช่วงแรกๆเป็นเพียงการใช้วงจรเกทในการทำงาน แต่ต่อมาเทคโนโลยีต่างๆพัฒนามากขึ้น เริ่มมีการสร้างไมโครคอนโทรลเลอร์ ณ จุดนี้ได้มีการใช้งานซอฟต์แวร์ (Software) ในการทำงาน แต่ซอฟต์แวร์ที่ใช้ยังไม่เป็นมาตรฐาน จึงได้เกิดแนวความคิดที่จะนำเอาระบบปฏิบัติการเข้าไปฝังไว้ในอุปกรณ์ต่างๆ เพื่อที่จะสามารถปรับปรุงแก้ไขโปรแกรมต่างๆบนอุปกรณ์เหล่านั้นได้ง่ายขึ้น จึงเป็นที่มาของระบบสมองกลฝังตัวดังกล่าว

ในอนาคตรบบเทคโนโลยีแบบฝังตัวจะมีส่วนเกี่ยวข้องกับชีวิตประจำวันของมนุษย์มากขึ้น ทั้งอุปกรณ์เครื่องใช้ภายในบ้านหรือในโรงงานอุตสาหกรรม ไม่ว่าจะเป็นโทรทัศน์ ตู้เย็น รถยนต์ หรือเครื่องมือเครื่องจักรต่างๆในโรงงานอุตสาหกรรม ซึ่งปัจจุบันโรงงานอุตสาหกรรมต่างๆได้นำเอาระบบสมองกลฝังตัวมาใช้อย่างกว้างขวาง ทำให้เครื่องมือเครื่องจักรเหล่านั้นมีความฉลาดและมีความสามารถเพิ่มมากขึ้น อย่างไรก็ตามอุปกรณ์ที่ใช้เทคโนโลยีระบบฝังตัวนั้นจะสามารถทำงานได้จะต้องมีการควบคุมหรือกำหนดคุณสมบัติก่อนการทำงาน ดังตัวอย่างเช่น เครื่องซักผ้าที่ทำงานได้ด้วยตัวเองจำเป็นต้องมีการควบคุมหรือกำหนดค่าคุณสมบัติของชนิดผ้าจึงจะทำให้เครื่องซักผ้าสามารถทำงานได้ด้วยตัวเอง เป็นต้น

จากคุณสมบัติของลินุกซ์ (Linux) ที่สามารถปรับแต่งเคอร์เนล (Kernel) ให้กระชับรัดกุมเหมาะสมกับการทำงานของอุปกรณ์ต่างๆ รวมถึงประสิทธิภาพในการใช้งานที่สามารถทำงานได้รวดเร็วและมีความเสถียรภาพ ทำให้บริษัทผู้ผลิตเครื่องมือเครื่องใช้ต่างๆ เริ่มนำลินุกซ์มาใช้งานเป็นระบบปฏิบัติการในเครื่องมือเครื่องใช้ของตัวเองมากขึ้น และไม่ใช่นำเอาลินุกซ์มาบรรจุในอุปกรณ์เท่านั้นที่น่าสนใจยังมีการนำลินุกซ์ไปประยุกต์ใช้กับระบบงานเฉพาะบางอย่างได้อย่างดีอีกด้วย

สำหรับปริญญานิพนธ์นี้มีแนวคิดมาจากโครงการ Trailblazer ของ Silver Jack Resort ซึ่งอยู่ในหนังสือ Embedded Linux®: Hardware, Software, and Interfacing ที่แต่งโดย Carig Hollabaugh ซึ่งเป็นโครงการสำหรับสร้างระบบจัดการการเล่นสกีทั้งหมด ตั้งแต่การควบคุมกระแสไฟฟ้า การจับเวลาการแข่งขันและการสร้างหิมะ เป็นต้น

ในส่วนของปริญญานิพนธ์นี้เป็นการจำลองการทำงานระบบเครน (Crane) ที่สามารถเคลื่อนที่ได้เพียงอย่างเดียวเพื่อใช้ทดสอบวิธีการใช้งาน Embedded Linux ในการควบคุมอุปกรณ์ที่ไม่มาตรฐาน และยังใช้เพื่อทดสอบความสามารถของ Embedded Linux ทั้งในด้านประสิทธิภาพในการทำงาน และความสามารถในการใช้งานกับฮาร์ดแวร์ (Hardware) ที่มีประสิทธิภาพต่ำ

1.2 วัตถุประสงค์

1. เพื่อศึกษาและทำความเข้าใจระบบที่ทำงานแบบฝังตัว (Embedded systems)
2. เพื่อทราบถึงประสิทธิภาพในการทำงานของ Embedded Linux System
3. เพื่อศึกษาและทำความเข้าใจการทำงานของ Linux Device Driver
4. เพื่อทำความเข้าใจการออกแบบการทำงานที่สอดคล้องกันระหว่างฮาร์ดแวร์และซอฟต์แวร์

1.3 ประโยชน์ที่คาดว่าจะได้รับ

1. สามารถนำ Embedded Linux System ไปประยุกต์ใช้งานกับอุปกรณ์ประมวลผลแบบอื่นได้
2. สามารถสร้าง Linux Device Driver ได้
3. สามารถออกแบบ Linux Device Driver ให้เหมาะสมกับฮาร์ดแวร์เฉพาะอย่างได้
4. สามารถนำหลักการการทำงานไปประยุกต์ใช้กับระบบงานที่มีขนาดใหญ่ได้

1.4 ทฤษฎีและแนวความคิดที่ใช้ในการทำงาน

ในปริญญานิพนธ์นี้ผู้จัดทำได้เลือกใช้ระบบปฏิบัติการ Debian Linux ซึ่งเป็นระบบปฏิบัติการประเภท Unix Clone Operating System ด้วยเหตุที่สามารถรองรับการทำงานกับอุปกรณ์ภายนอกได้มากมายหลายชนิดและจัดหาได้ง่าย ซึ่งจากการทดสอบการทำงานของ Debian Linux เปรียบเทียบกับระบบปฏิบัติการ Unix Clone Operating System ตระกูลอื่นพบว่า Debian Linux สามารถนำไปใช้ในการพัฒนาโปรแกรมได้ดีกว่า ซึ่งอาจมีสาเหตุเนื่องมาจาก Debian Linux ไม่ได้เปลี่ยนแปลงโครงสร้างของไบนารี (Library) และ ฟังก์ชัน (Function) พื้นฐานของลินุกซ์ จึงทำให้ Debian Linux มีความเสถียรภาพและเกิดปัญหาในการใช้น้อย

1.5 ขอบเขตของงานวิจัย

1. ระบบจะต้องทำงานแบบฝังตัว
2. โปรแกรมหลักจะต้องถูกสร้างโดยมีลักษณะเป็น Linux module
3. เครื่องจะต้องสามารถสั่งงานได้จากเครื่องคอมพิวเตอร์ระยะไกล
4. ส่วนติดต่อกับผู้ใช้สามารถใช้ได้ทั้งเว็บ (Web) และคอมมานด์เชลล์ (Command shell)
5. ตัวเครื่องสามารถเคลื่อนที่ได้ทั้งในแนวนอนและแนวตั้ง
6. เครื่องถูกทำให้เคลื่อนที่โดยใช้สเต็ปมิงมอเตอร์ (Stepping motor)
7. ระบบจะต้องสามารถแก้ไขปัญหาที่อาจเกิดขึ้นด้วยตัวเองได้ในระดับหนึ่ง
8. ระบบจะต้องมีส่วนแสดงสถานะของระบบในระหว่างการทำงาน

1.6 วิธีการดำเนินงาน

การทำงานสามารถแบ่งเป็นช่วงหลักๆได้ ดังนี้

1.6.1 ศึกษาข้อมูล

- ศึกษาการทำงานของ Linux Kernel
- ศึกษาวิธีการเขียน Linux Device driver

1.6.2 ออกแบบระบบ

1.6.2.1 กำหนดขอบเขตของโครงการ

1.6.2.2 กำหนดโครงสร้างและการทำงานของระบบโดยรวม

1.6.2.3 ออกแบบโปรแกรม

- คุณสมบัติของโปรแกรม
- โครงสร้างของโปรแกรม
- จำนวนและขนาดของข้อมูลที่ใช้งาน
- สเตทไดอะแกรม (state diagram)
- ซีควเอนซ์ไดอะแกรม (sequence diagram)
- โฟลว์ชาร์ท (flowchart)

1.6.2.4 เลือกเซิร์ฟเวอร์ (Server)

- คุณสมบัติของฮาร์ดแวร์
- คุณสมบัติของซอฟต์แวร์

1.6.2.5 ออกแบบ Crane interface

- คุณสมบัติของ crane interface
- บล็อกไดอะแกรม (block diagram)
- ออกแบบวงจร

1.6.2.6 ออกแบบครนและ โครง (Rack)

- คุณสมบัติของครนและ โครง
- ออกแบบโครงสร้างของครน
- ออกแบบโครงสร้างของ โครง

1.6.3 ติดตั้งและปรับแต่งเซิร์ฟเวอร์

- ติดตั้ง Debian Linux v3.2
- ปรับแต่งระบบ
- ทดสอบการทำงาน

1.6.4 เขียนโปรแกรม

- โปรแกรมควบคุมครน
- โปรแกรมติดต่อกับผู้ใช้งานเว็บ
- ทดสอบการทำงาน

1.6.5 จัดทำส่วนติดต่อกับเครน

- เตรียมอุปกรณ์
- ประกอบวงจรส่วนติดต่อกับเครน
- ทดสอบการทำงาน

1.6.6 จัดทำเครนและโครง

- เตรียมอุปกรณ์
- ประกอบโครง
- ประกอบเครน
- ทดสอบการทำงาน

1.6.7 ผลการทดสอบระบบโดยรวม

- ปัญหาเกี่ยวกับเซิร์ฟเวอร์
- ปัญหาเกี่ยวกับโปรแกรม
- ปัญหาเกี่ยวกับส่วนติดต่อกับเครน
- ปัญหาเกี่ยวกับเครนและโครง

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

จากการศึกษาข้อมูลเบื้องต้นพบว่าระบบสมองกลฝังตัวไม่มีการแบ่งประเภทที่แน่นอน โดยในแต่ละบริษัทผู้ผลิตจะมีการแบ่งประเภทแตกต่างกัน และส่วนใหญ่จะแบ่งประเภทตามความสามารถของอุปกรณ์ที่ใช้ในระบบ ดังนี้

2.1 ระบบที่ทำงานด้วยไมโครคอนโทรลเลอร์ (Microcontrollers)

ส่วนใหญ่เป็นระบบที่ถูกออกแบบให้ทำงานเฉพาะอย่าง โดยมีการใช้ไมโครคอนโทรลเลอร์ขนาดเล็กเป็นตัวควบคุมและส่วนใหญ่ไมโครคอนโทรลเลอร์จะถูกกำหนดการทำงานมาจากบริษัทผู้ผลิตโดยตรง ผู้ใช้ไม่สามารถทำการโปรแกรมการทำงานตามความต้องการได้ ลักษณะพิเศษอีกประการหนึ่งคือระบบประเภทนี้จะไม่ใช้ฐานเวลาจริง (Real-time clock) ในการทำงาน แต่มีข้อดีประการหนึ่งคือมีความเสถียรภาพในการทำงานจากการสำรวจพบว่ามีอัตราการทำงานไม่สำเร็จ (Failure rate) เท่ากับ 1 ครั้งในการทำงานทั้งหมด 100,000 ครั้ง (ข้อมูลจาก www.icee.org)

2.2 ระบบที่ทำงานด้วยไมโครโพรเซสเซอร์ (Microprocessors)

ไมโครโพรเซสเซอร์เป็นอุปกรณ์ที่มีความสามารถสูงกว่าไมโครคอนโทรลเลอร์ นอกจากความเร็วในการทำงานที่มีมากกว่าแล้วไมโครโพรเซสเซอร์ยังสามารถทำการโปรแกรมได้โดยเก็บคำสั่งไว้ที่อุปกรณ์จำพวก ROM หรือ EPROM ซึ่งทำให้สะดวกในการใช้งาน แต่ไมโครโพรเซสเซอร์ยังมีปัญหาในการทำงานร่วมกับโปรแกรมที่ต้องการใช้งานเทียบกับฐานเวลาจริงและมีอัตราการทำงานไม่สำเร็จประมาณ 2 เปอร์เซ็นต์ (ข้อมูลจาก www.icee.org)

2.3 ระบบขนาดใหญ่ (Large-scale systems)

ระบบนี้เกิดจากการทำงานระหว่างไมโครโพรเซสเซอร์หรือไมโครคอนโทรลเลอร์ตั้งแต่ 1 ตัวขึ้นไปตัวอย่างเช่น คอมพิวเตอร์หนึ่งเครื่องจะประกอบด้วยไมโครโพรเซสเซอร์ที่ต่อใช้งานร่วมกับฮาร์ดดิสก์ (Harddisk) ซึ่งมีไมโครคอนโทรลเลอร์เป็นของตัวเอง เป็นต้น ระบบนี้จะมีการใช้งานอ้างอิงกับฐานเวลาจริงทำให้ไม่เกิดปัญหาเกี่ยวกับเวลาขณะเขียนโปรแกรม แต่จะมีข้อเสีย คือมีอัตราการทำงานไม่สำเร็จสูงถึงประมาณ 35 เปอร์เซ็นต์

ซอฟต์แวร์ที่ใช้งานในระบบนี้จะแบ่งออกเป็น 2 ส่วนคือ ระบบปฏิบัติการและโปรแกรมควบคุม ถึงแม้แต่ละบริษัทผู้ผลิตฮาร์ดแวร์จะผลิตฮาร์ดแวร์ที่มีประสิทธิภาพและความสามารถแตกต่างกัน แต่สามารถใช้งานระบบปฏิบัติการชนิดเดียวกันได้ทำให้เกิดความคล่องตัวในการใช้งานมากขึ้น ดังนั้นเราสามารถแยกประเภทย่อยตามระบบปฏิบัติการที่ใช้งานได้ดังนี้

2.3.1 Unix Operating System

เป็นระบบปฏิบัติการที่อยู่ในรูปแบบการคำนวณสามารถทำงานได้ดีมากระบบหนึ่งเป็นระบบปฏิบัติการที่นำมาใช้งานแบบฝังตัวในช่วงแรกๆ มีข้อเสียคือ การกำหนดว่าจะนำไปใช้งานกับอุปกรณ์ใดต้องมีการกำหนดตั้งแต่เริ่มสร้างระบบปฏิบัติการ เพราะโครงสร้างของระบบส่วนใหญ่จะเป็นแบบ Monolithic ตัวอย่างเช่น Solaris, Sco Unix และ BSD Unix เป็นต้น

2.3.2 Unix Clone Operating System

เป็นระบบปฏิบัติการรุ่นใหม่ที่มีโครงสร้างการทำงานคล้ายกับ Unix แต่แตกต่างกันตรงโครงสร้างของระบบภายในจะเป็นแบบเคอร์เนลที่มีแกนระบบปฏิบัติการเพียงหนึ่งเดียวแต่สามารถนำส่วนทำงานย่อย (Module) ต่างๆเข้าไปผนวกเข้ากับระบบได้แม้ขณะกำลังทำงานอยู่จึงทำให้เกิดความคล่องตัวในการใช้งานและเป็นที่ยอมรับอยู่ในขณะนี้ ตัวอย่างเช่น GNU Debian Linux , Free BSD และ Slackware Linux เป็นต้น

2.3.3 Windows Operating System

เป็นระบบที่ถูกพัฒนาขึ้นโดยมีวัตถุประสงค์เพื่อให้่ายต่อการใช้งาน ส่วนใหญ่นำไปใช้งานกับอุปกรณ์จำพวก Pocket PC ข้อดีคือสวยงามและใช้งานง่าย ข้อเสียคือขณะทำงานจะใช้ทรัพยากรของระบบมากทำให้ไม่เหมาะกับการนำมาใช้งานในระบบที่ต้องการความกระชับและรวดเร็ว ตัวอย่างเช่น Palm OS , Windows CE เป็นต้น

บทที่ 3

วิธีดำเนินการทำงาน

3.1 บทนำ

เนื่องจากปัญญานิพนธ์ทั้งหมดประกอบด้วยส่วนการทำงานหลัก 3 ส่วนได้แก่ ส่วนโปรแกรม ส่วนวงจรอิเล็กทรอนิกส์ และส่วนงานทางกล เห็นได้ว่าต้องใช้ความรู้ในการทำงานหลายด้าน ดังนั้น การปรับตัวในการทำงานที่หลากหลายจึงเป็นสิ่งจำเป็น นอกเหนือจากการแบ่งงาน และแบ่งเวลาที่เหมาะสมแล้ว ผู้จัดทำได้แบ่งขั้นตอนการทำงานที่เหมาะสม ดังต่อไปนี้

3.2 การศึกษาข้อมูล

3.2.1 ศึกษาการทำงานของ Linux kernel

Linux kernel เป็น โปรแกรมหลักของระบบที่ทำหน้าที่ดูแลการใช้งานโปรแกรมและฮาร์ดแวร์ (Hardware) ต่างๆ ของผู้ใช้ให้เป็นระเบียบและถูกต้อง โครงสร้างภายในของ Linux Kernel ประกอบด้วย ส่วนต่างๆ ดังนี้

- **Process management**

ทำหน้าที่สร้างโปรเซส (Process) ที่เข้ามาใหม่และดูแลโปรเซสที่กำลังทำงานอยู่และทำลายโปรเซสที่ไม่ต้องการในระบบ

- **Memory management**

ทำหน้าที่ดูแลและจัดการการใช้งานหน่วยความจำ (Memory) ของโปรเซสทั้งหมดไม่ให้เกิดการแย่งใช้งานหรือเลื่อมล้ำพื้นที่ทำงาน

- **File system**

ลินุกซ์จะมองทุกอย่างเป็นไฟล์ (File) และสามารถใช้งานกับระบบไฟล์ได้หลายระบบบนลินุกซ์ ดังนั้นจึงต้องมีส่วนที่ดูแลจัดการระบบไฟล์ดังกล่าว

- **Device control**

เป็นส่วนที่ทำหน้าที่ดูแลการใช้งานอุปกรณ์ภายนอกทั้งหมด ตั้งแต่ฟลอปปีดิสก์ (Floppy Disk) ไปจนถึงปริ้นเตอร์ (Printer) เพราะลินุกซ์จะไม่ยอมให้ผู้ใช้สามารถใช้งานอุปกรณ์ภายนอกได้โดยตรงเพื่อป้องกันการยึดครองอุปกรณ์ ดังนั้นเคอร์เนลจึงมีระบบที่คอยจัดการการทำงานดังกล่าว

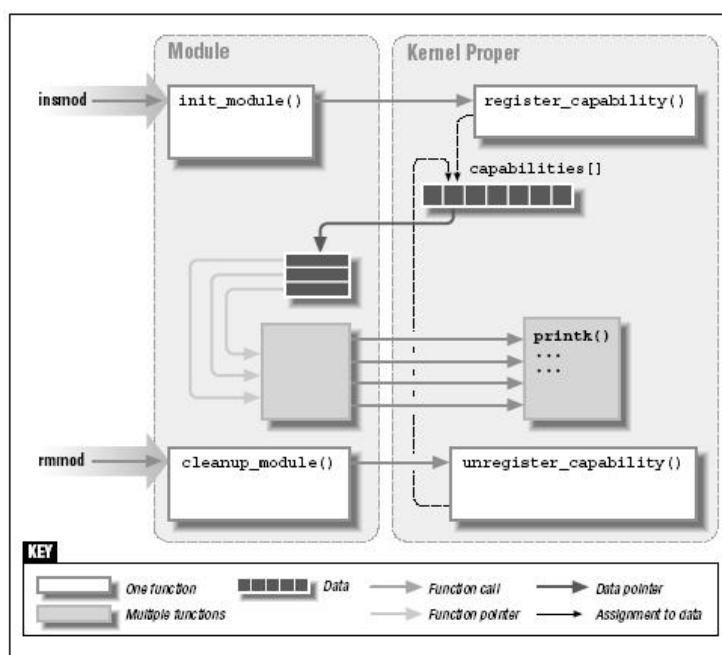
- **Networking**

เป็นอีกส่วนหนึ่งที่ระบบปฏิบัติการต้องดูแล เพราะข้อมูลที่เข้าออกผ่านระบบเน็ตเวิร์ก (Network) จะเป็นแบบอะซิงโครนัส (Asynchronous) ดังนั้นจึงต้องมีการติดตามดูแลการทำงานของแพ็กเกจ (Packet) ตลอดเวลา

หมายเหตุ สามารถหารายละเอียดของเคอร์เนลเพิ่มเติมได้จาก www.kernel.org

3.2.2 ศึกษาวิธีการเขียน Linux Device driver

เหตุผลที่ต้องใช้ Linux Device driver นั้นผู้ใช้ควรทราบก่อนว่าลินุกซ์จะแบ่งพื้นที่การทำงานออกเป็น 2 ส่วน คือ user space และ kernel space ซึ่งผู้ที่ใช้งานอยู่บน user space จะไม่สามารถเข้าใช้งานโปรแกรมบน kernel space ได้โดยตรง การใช้งานจะต้องกระทำผ่าน system call เท่านั้น ดังนั้นการสร้างแอปพลิเคชัน (Application) ที่สามารถแบ่งปันทรัพยากรของระบบได้นั้นจะต้องสร้างขึ้นภายใน kernel space โดยการสร้างโมดูล (Module) ที่ทำงานตามที่ต้องการจากนั้นนำโมดูลที่ได้ไปวางไว้ในระดับ kernel space เพื่อผนวกเข้ากับเคอร์เนลโดยการใช้คำสั่ง insmod เพียงเท่านั้นโปรแกรมที่สร้างขึ้นจะสามารถรองรับการทำงานให้กับยูสเซอร์ (User) ทุกคนได้ ซึ่งการนำโมดูลเข้าและออกนั้นสามารถแสดงได้ดังรูป

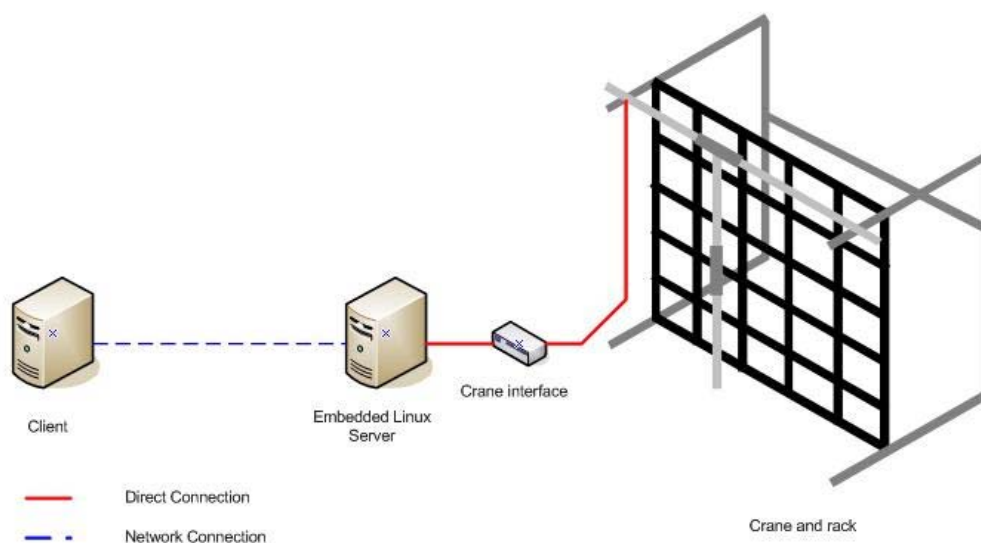


รูปที่ 3-1 แสดงการทำงานของคำสั่ง insmod และ rmmod

ในรูปที่ 3-1 แสดงให้เห็นว่าเมื่อมีการใช้คำสั่ง insmod จะมีการรันฟังก์ชัน (run function) init_module() ของโมดูลเพื่อทำงานบางอย่าง เช่น ตรวจสอบว่ามีทรัพยากรว่างหรือไม่ หรือแสดงสถานะของโมดูลก่อนถูกใช้งาน เป็นต้น

3.3 การออกแบบระบบ

3.3.1 โครงสร้างและการทำงานของระบบโดยรวม



รูปที่ 3-2 แสดงโครงสร้างโดยรวมของระบบ

- **Client**

เป็นส่วนที่ผู้สเซอร์สามารถสั่งงานได้จากระยะไกลมายังเซิร์ฟเวอร์ซึ่งสามารถมีได้มากกว่า 1 เครื่อง แต่ไม่สามารถสั่งงานได้พร้อมกันและใช้ได้กับเครื่องคอมพิวเตอร์ที่สามารถเปิดเว็บเบราว์เซอร์ (Web Browser) และรองรับการทำงานระบบเน็ตเวิร์ก

- **Embedded Linux Server**

เป็นส่วนควบคุมหลักของระบบทั้งหมดและในการทดลองนี้จะให้ความสำคัญกับส่วนนี้เป็นพิเศษ โดยเซิร์ฟเวอร์จะถูกออกแบบให้ทำงานบนระบบปฏิบัติการลินุกซ์และถูกปรับแต่งให้ทำงานแบบ Embedded system ซึ่งโครงสร้างภายในจะใช้เครื่องคอมพิวเตอร์ที่มีสถาปัตยกรรมตระกูล x86

- **Crane interface**

เป็นส่วนที่ติดต่อระหว่างเซิร์ฟเวอร์กับอุปกรณ์ที่ติดตั้งบนเครนทั้งหมดทำหน้าที่แปลงระดับสัญญาณระหว่างดิจิทัล (Digital) ให้เป็นระดับแรงดันที่อุปกรณ์ภายนอกต้องการ หรือทำหน้าที่แปลงระดับแรงดันที่เกิดขึ้นภายนอกให้อยู่ในระดับสัญญาณทีแอล (TTL) เพื่อป้อนให้กับพอร์ตขนาน (Parallel port) เป็นต้น

- **Crane and Rack**

เป็นส่วนที่ทำหน้าที่หลักทางกลโดยโครงสร้างหลักจะประกอบด้วยเครนและโครง (Rack) ซึ่งเครนจะเป็นส่วนที่เคลื่อนที่ได้ทั้งแนวตั้งและแนวนอน อีกส่วนคือโครงจะเป็นส่วนที่ทำหน้าที่เสมือนชั้นวางของ หรืออาจนำไปประยุกต์แทนการใช้งานด้านอื่นได้

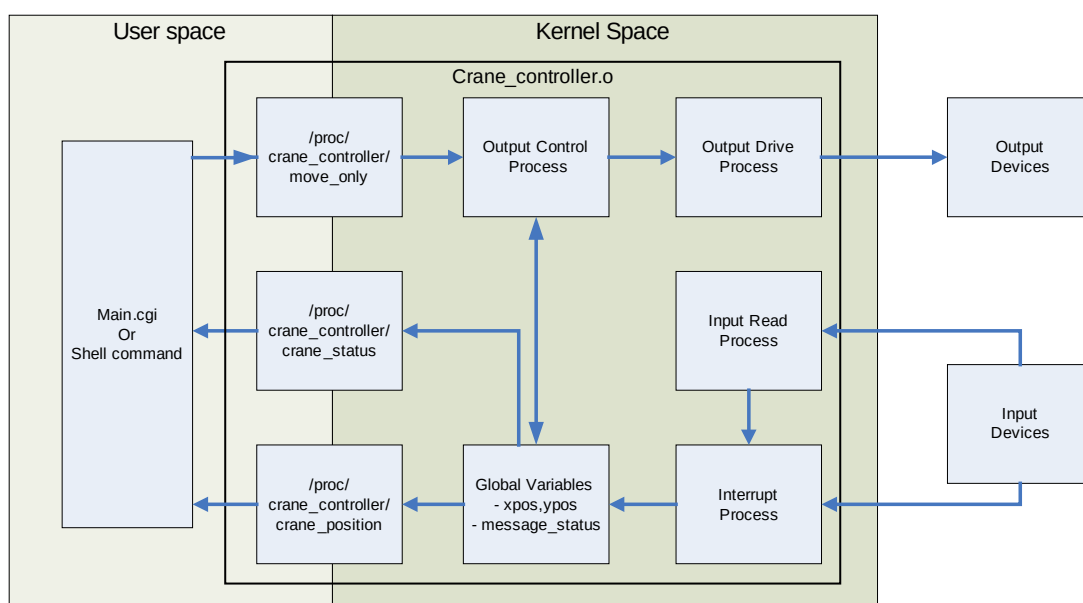
3.3.2 การออกแบบโปรแกรม

3.3.2.1 คุณสมบัติของโปรแกรม

- โปรแกรมจะถูกเขียนในลักษณะโมดูลเพื่อสามารถทำงานในระดับเคอร์เนลได้
- เมื่อเปิดเครื่องโปรแกรมจะต้องถูกติดตั้งโดยอัตโนมัติ
- โปรแกรมจะมีระบบตรวจสอบการทำงานด้วยตัวเองและสามารถแก้ไขปัญหาเบื้องต้นได้
- การติดต่อระหว่าง user space และ kernel space จะกระทำผ่าน Process file

3.3.2.2 โครงสร้างของโปรแกรม

เนื่องจากโปรแกรมมีลักษณะเป็นโมดูลจึงมีโครงสร้างที่คาบเกี่ยวระหว่าง user space และ kernel space ของระบบปฏิบัติการ โดยส่วนที่อยู่ใน user space นั้นผู้ใช้จะสามารถมองเห็นได้โดยตรงและถูกใช้เป็นเส้นทางการส่งและรับข้อมูลระหว่างผู้ใช้กับโมดูลที่กำลังทำงานอยู่ โครงสร้างหลักๆ ของโปรแกรมสามารถแสดงได้ดังรูป



รูปที่ 3-3 แสดงลักษณะโครงสร้างของ Crane controller device driver

- **User interface (Main.cgi or shell command)**

ทำหน้าที่ติดต่อกับผู้ใช้งาน เป็นส่วนรับคำสั่งจากยูสเซอร์แล้วส่งข้อมูลไปยัง Process file ในทางกลับกันยังทำหน้าที่รับข้อมูลจาก Process file มาแสดงยังยูสเซอร์ อีกทางหนึ่งด้วย โดยช่องทางการใช้งานระบบสามารถทำได้ 2 ทางคือ Linux shell command และหน้าเว็บเพจ (Web page) main.cgi

- **/proc/crane_controller/move_only**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โมดูลถูกโหลด (Load) เข้าสู่เคอร์เนลทำหน้าที่เป็นส่วนรับค่าตำแหน่งของเครนจากผู้ใช้และส่งค่าตำแหน่งต่อไปยังส่วนควบคุมการหมุนของ สเต็ปป์มอเตอร์ภายในของโปรเซสต่อไป

- **/proc/crane_controller/crane_position**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โมดูลถูกโหลดเข้าสู่เคอร์เนล ทำหน้าที่นำค่าตัวแปรโกลบอล (Global) ตำแหน่งของเครนไปแสดงยัง user space

- **/proc/crane_controller/crane_status**

เป็น Process file ที่ถูกสร้างขึ้นขณะที่โปรแกรมทำงาน ทำหน้าที่นำค่าตัวแปรโกลบอลของสถานะเครนไปแสดงยัง user space

- **output control process**

เป็นโปรเซสที่อยู่ภายในโมดูลทำหน้าที่รับคำสั่งให้สเต็ปป์มอเตอร์หมุนนำเครนไปยังตำแหน่งที่ผู้ใช้งานต้องการ ซึ่งโปรเซสนี้จะเป็นตัวควบคุมทิศทางการหมุนตลอดจนความเร็วของสเต็ปป์มอเตอร์ทั้งหมด โดยจะทำการตรวจสอบค่าตำแหน่งของเครนจากตัวแปรโกลบอลตลอดเวลาเพื่อที่จะทราบว่าเครนไปถึงตำแหน่งที่ต้องการแล้วหรือยัง

- **output drive process**

เป็นส่วนที่ทำหน้าที่จัดตำแหน่งข้อมูลที่จะส่งให้กับสเต็ปป์มอเตอร์ โดยจะใช้หลักการของการมาสกิ้งบิต (Masking bit) ข้อมูลเพื่อกำหนดให้ค่าส่งไปยังสเต็ปป์มอเตอร์ได้ถูกตัว และหมุนได้อย่างถูกต้อง โดยจะมีการอ่านค่าการหมุนจากสเต็ปเทเบิล (Step table) ในส่วนของตัวแปรโกลบอล

- **input read process**

เป็นส่วนที่ทำหน้าที่รับสัญญาณดิจิทัลจากอุปกรณ์ภายนอกอย่างต่อเนื่องโปรเซสนี้จะต้องทำงานสอดคล้องกับอินเทอร์รัพโปรเซส (Interrupt process) เพราะเมื่อมีการเปลี่ยนแปลงของสัญญาณอินพุต (Input) สัญญาณอินเทอร์รัพ (Interrupt) จะต้องเปลี่ยนแปลงด้วย

- **interrupt process**

เป็นส่วนที่ทำหน้าที่ทำงานตามสัญญาณอินเทอร์รัพที่เกิดขึ้นจากอุปกรณ์ภายนอก โดยสัญญาณอินเทอร์รัพจะเกิดขึ้นพร้อมๆกับการเปลี่ยนแปลงของข้อมูลอินพุต ดังนั้นเมื่อเกิดอินเทอร์รัพขึ้นจะต้องมีการอ่านค่าจาก input read process เพื่อที่จะทราบว่าสัญญาณอินเทอร์รัพของข้อมูลใด

3.3.2.3 จำนวนและขนาดของข้อมูลที่ใช้งาน

รายละเอียดของสัญญาณอินพุต

● TOP limit switch	1	Bit
● BOTTOM limit switch	1	Bit
● LEFT limit switch	1	Bit
● RIGHT limit switch	1	Bit
● CRANE Column sensor	1	Bit
● CRANE ROW sensor	1	Bit

รายละเอียดของสัญญาณเอาต์พุต (Output)

● X STEPPER MOTOR Drive	4	Bit
● Y STEPPER MOTOR Drive	4	Bit

รายละเอียดของพอร์ตขนานที่ใช้

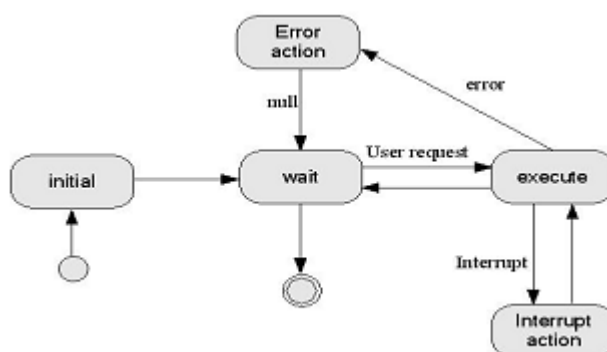
ขาที่ใช้รับข้อมูลอินพุต

● Bit 0	crane coulumn sensor
● Bit 1	crane row sensor
● Bit 2	not use
● Bit 3	not use
● Bit 4	right limit switch
● Bit 5	top limit switch
● Bit 6	left limit switch
● Bit 7	bottom limit switch

ขาที่ใช้ส่งข้อมูลเอาต์พุต

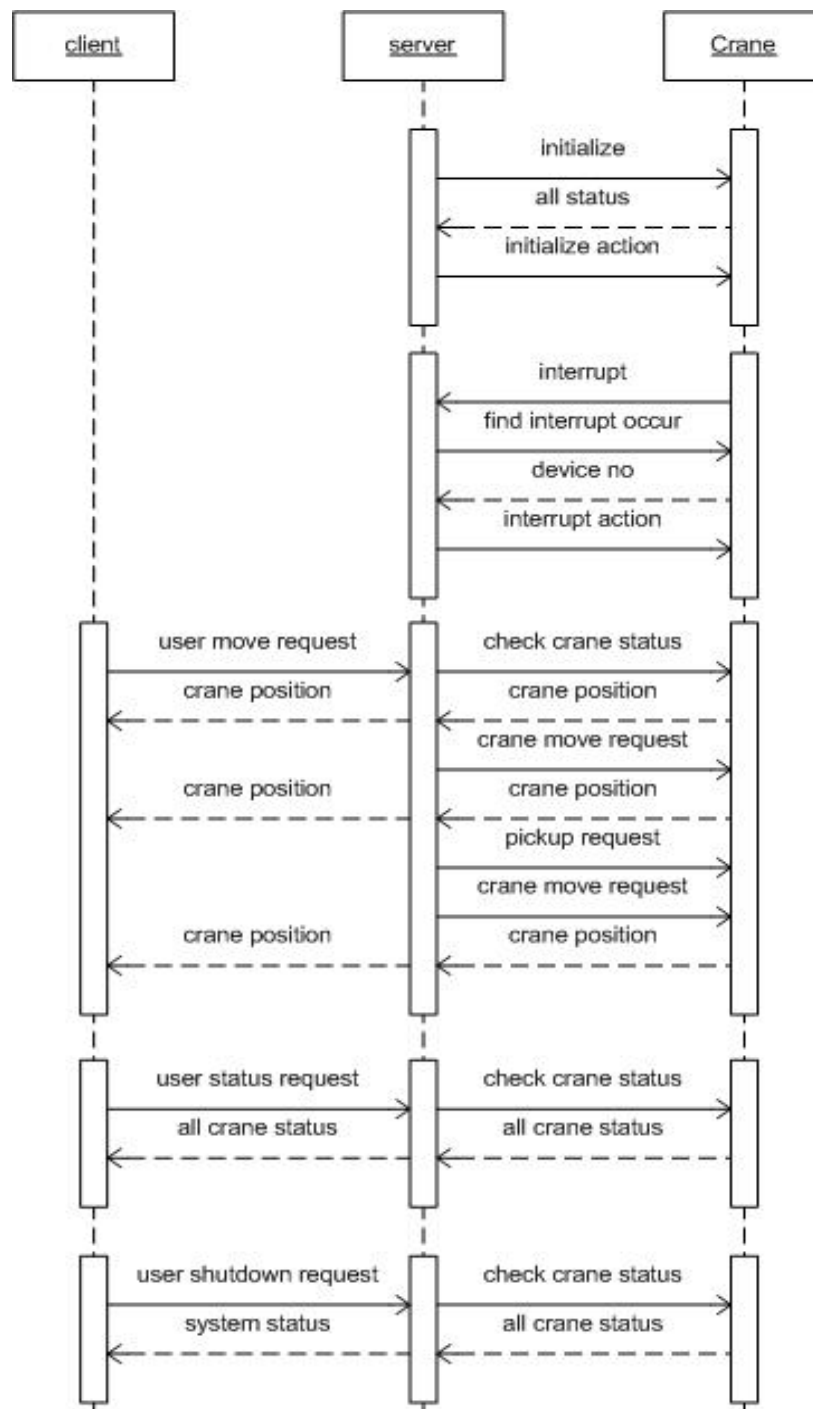
● Bit 0-3	x motor
● Bit 4-7	y motor

3.3.2.4 state diagram



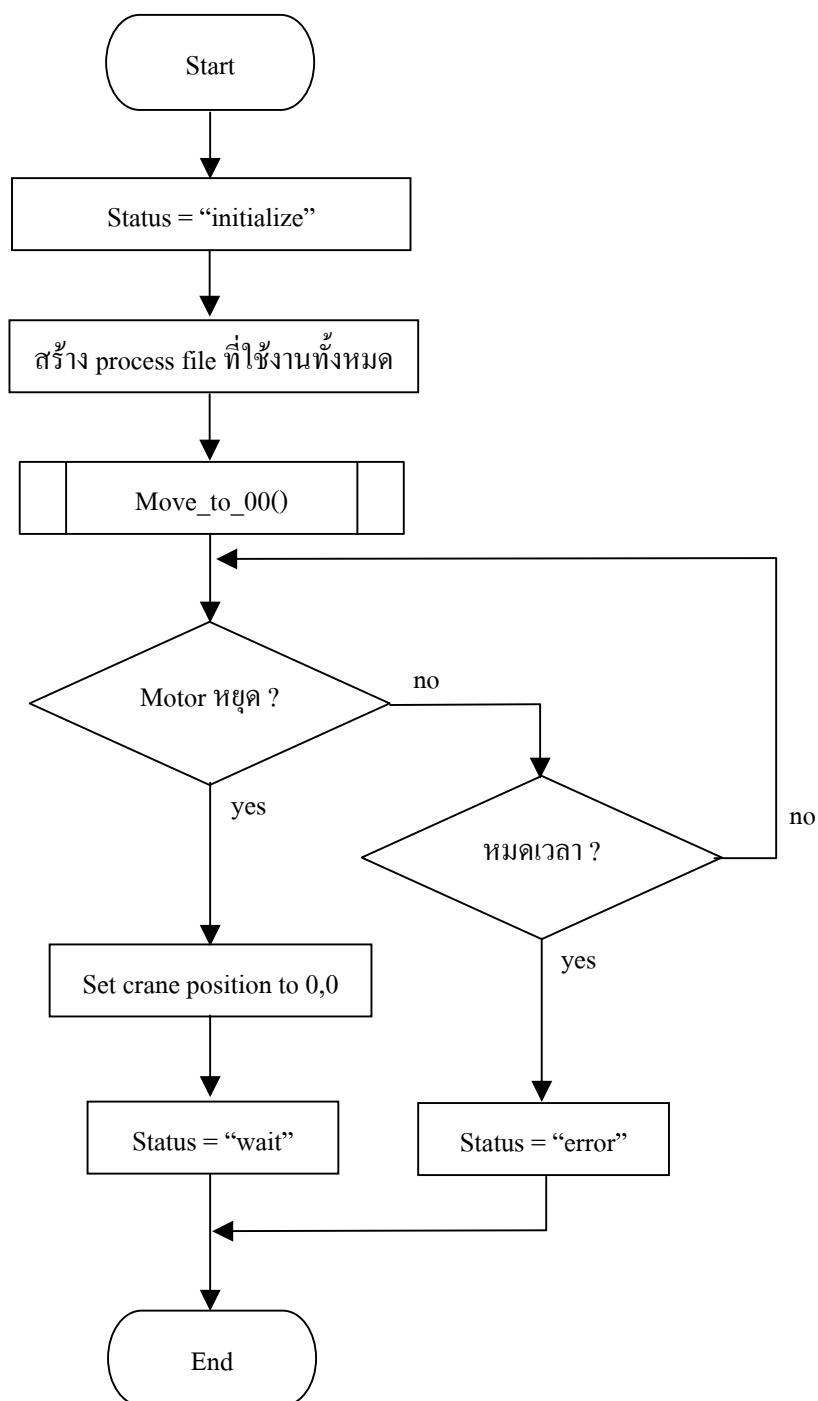
รูปที่ 3-4 แสดง state diagram ของระบบ

3.3.2.5 sequence diagram

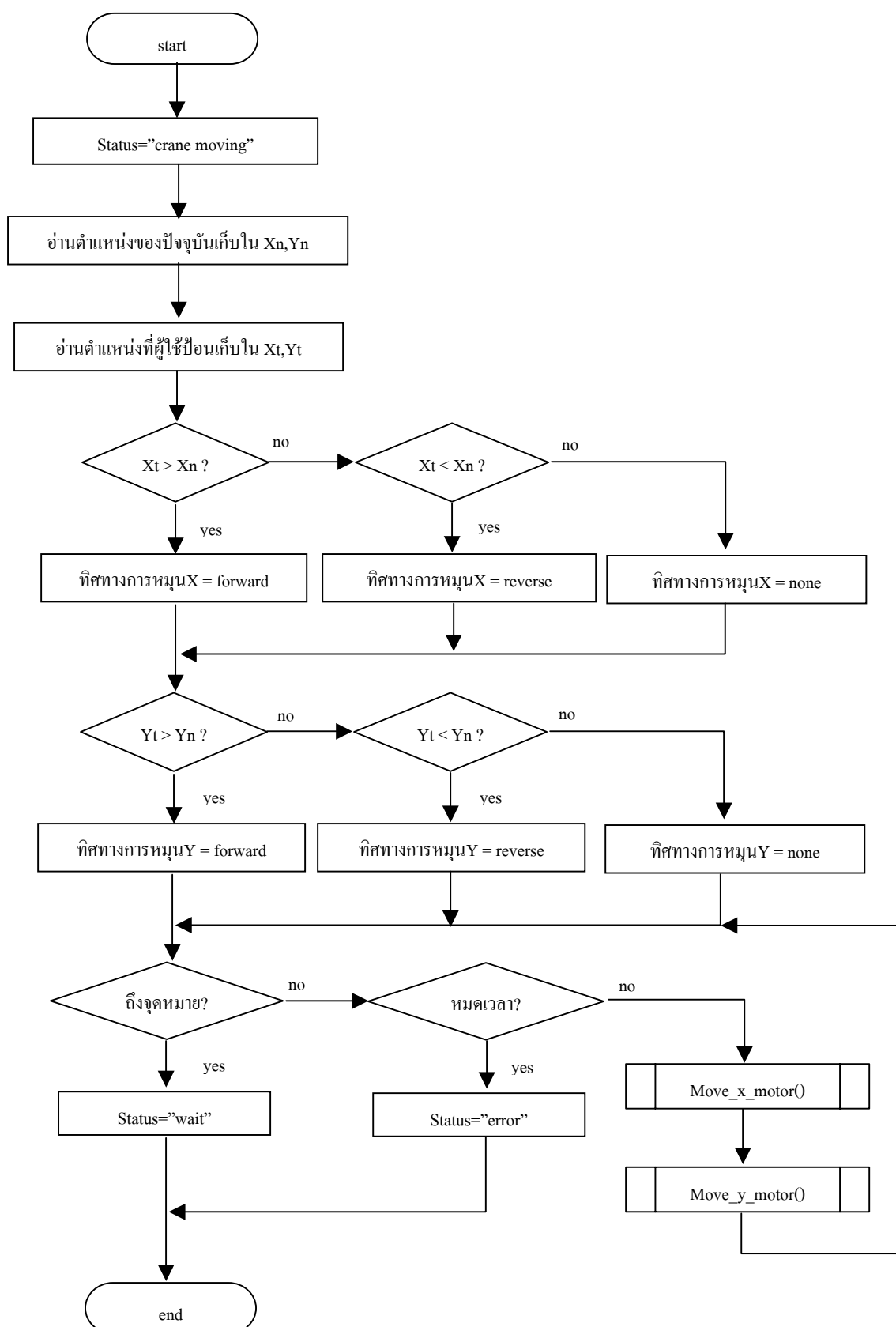


รูปที่ 3-5 แสดง sequence diagram ของระบบ

3.3.2.6 flowchart

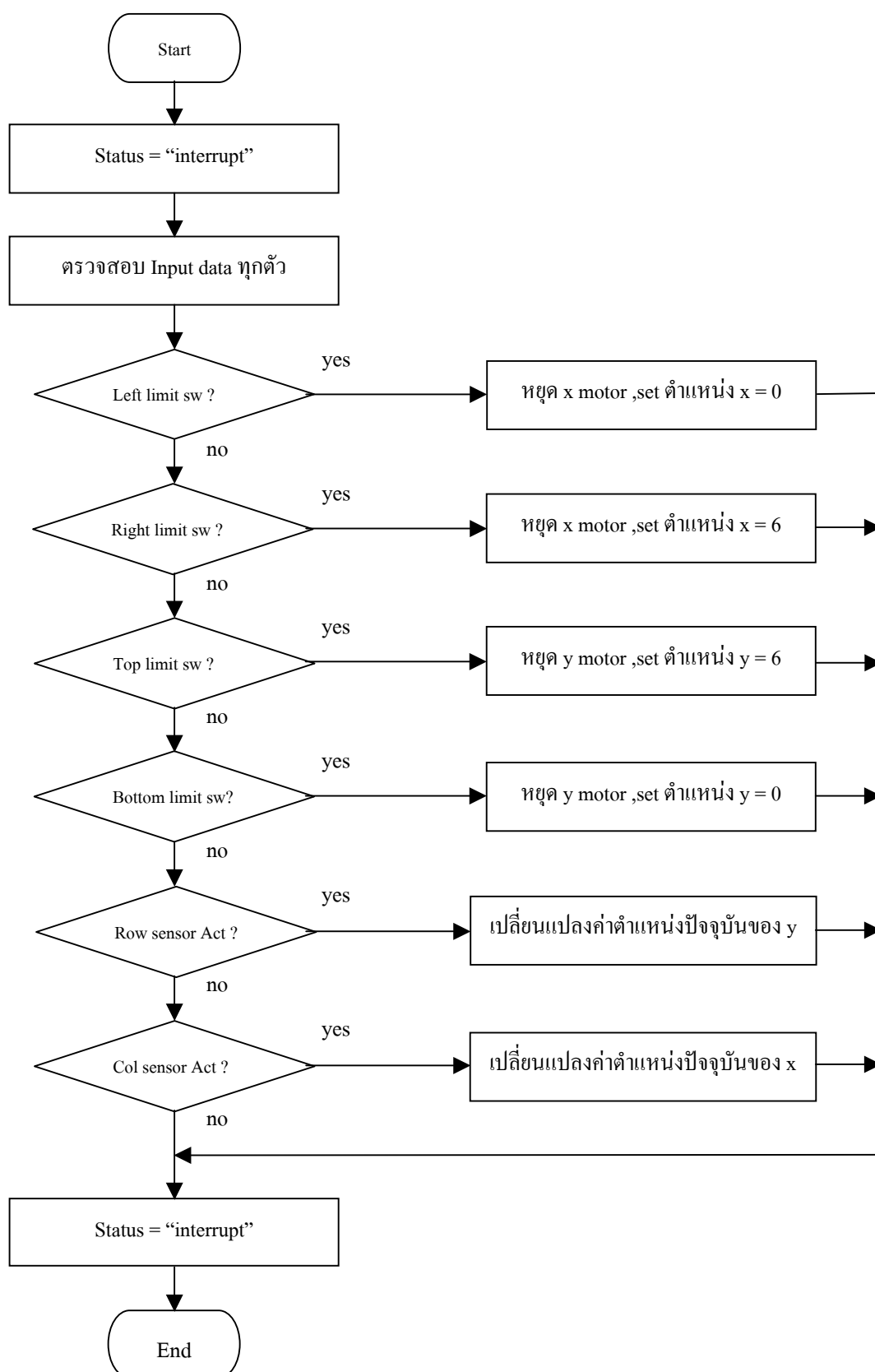
รูปที่ 3-6 แสดงขั้นตอนการทำงานของฟังก์ชัน `init_module()`

3.3.2.7 Crane_move procedure



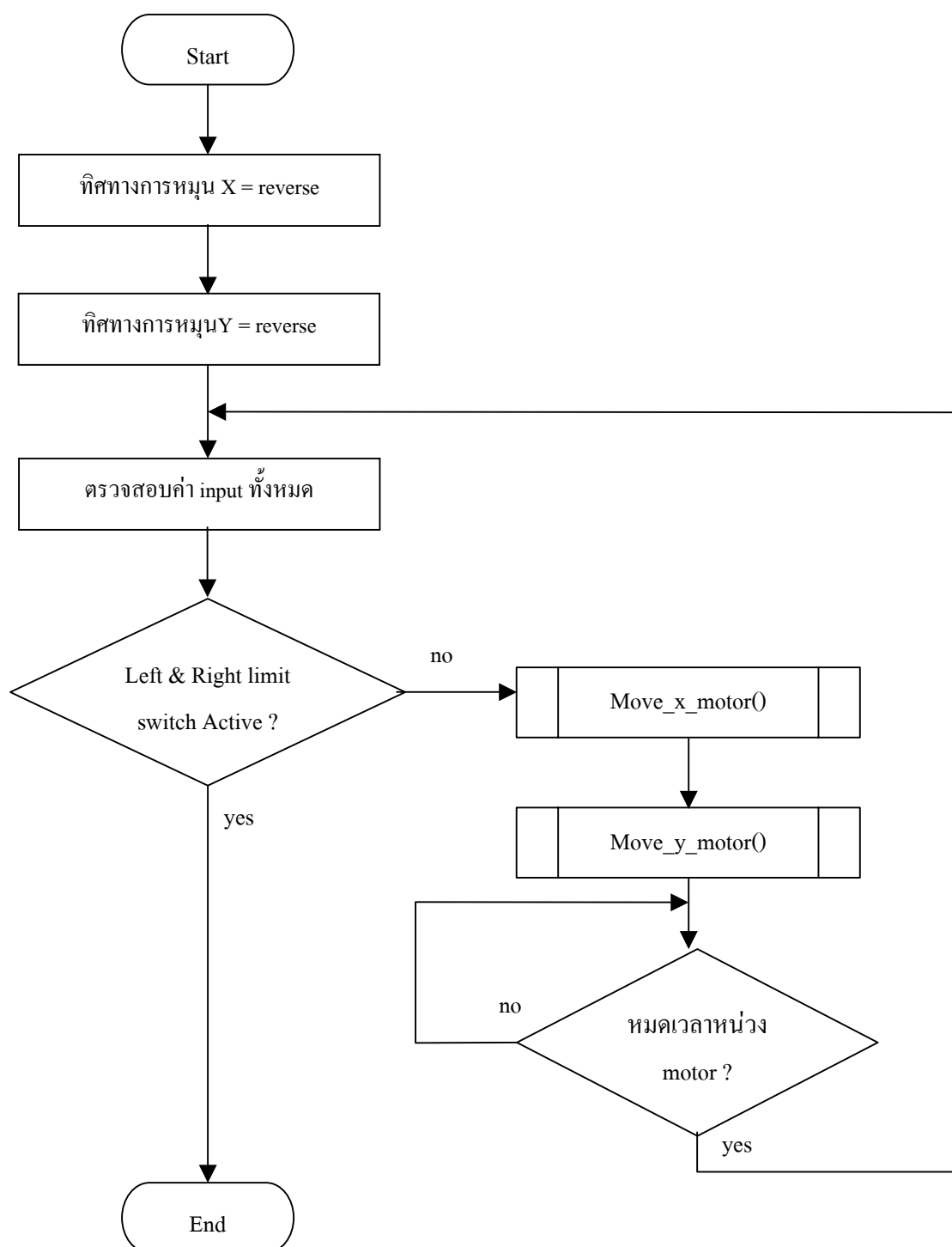
รูปที่ 3-7 แสดงขั้นตอนการทำงานเมื่อมีการสั่งให้เครนเคลื่อนที่

3.3.2.8 Interrupt_action procedure



รูปที่ 3-8 แสดงขั้นตอนการทำงานเมื่อมีอินเตอร์รัพ

3.3.2.9 Move to 0,0 position



รูปที่ 3-9 แสดงขั้นตอนการทำงานของ Move_to_00 ()

3.3.3 การเลือก Embedded Linux server

เนื่องจากการทำงานแบบ Embedded นั้น ไม่ได้กำหนดประเภทหรือขนาดของเมนบอร์ด (Mainboard) ที่ใช้ในการทำงาน ดังนั้นทางผู้จัดทำจึงได้เลือกเมนบอร์ดคอมพิวเตอร์ตระกูล x86 เพราะ

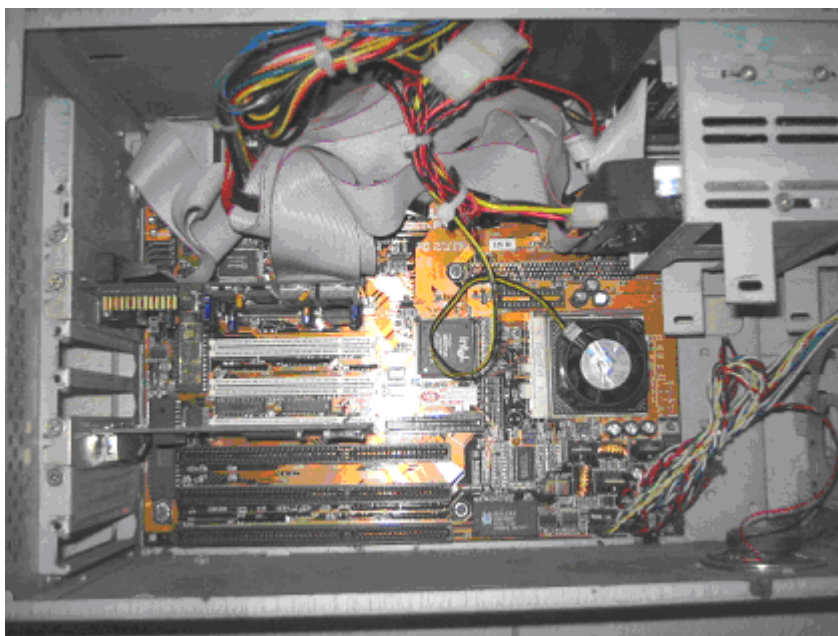
สามารถมีคุณสมบัติที่ครอบคลุมกับการทำงานของระบบและสามารถจัดหาได้ง่าย และเพื่อความสะดวกในการทำงาน โดยคุณสมบัติของเครื่องคอมพิวเตอร์ที่ผู้จัดทำเลือกใช้มีรายละเอียดดังนี้

3.3.3.1 Hardware Specifications

- CPU Pentium 200 MMX
- Memory 64 MB
- Hard Disk ขนาด 8 GB
- Card LAN 10/100
- VGA Card PCI

3.3.3.2 Software Specifications

- Debian Linux v. 3.0 r2



รูปที่ 3-10 แสดงเครื่องคอมพิวเตอร์ที่ทำหน้าที่ *Embedded Linux Server*

3.3.4 การออกแบบวงจรเรณอินเตอร์เฟส

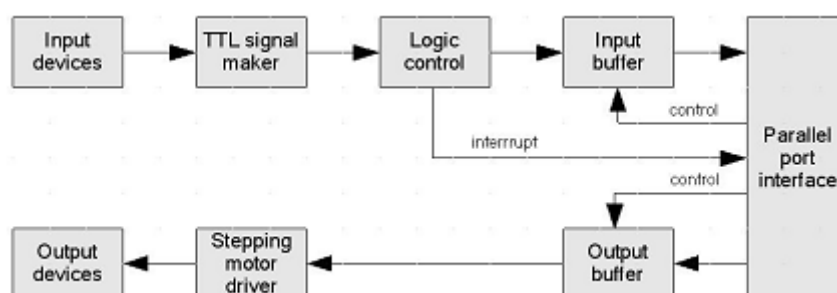
เรณอินเตอร์เฟสเป็นวงจรอิเล็กทรอนิกส์ (Electronic) ที่ทำหน้าที่ติดต่อระหว่างเซิร์ฟเวอร์กับอุปกรณ์ที่ติดตั้งบนเรณทั้งหมด เนื่องจากอุปกรณ์ภายนอกจะใช้ระดับสัญญาณแตกต่างจากสัญญาณดิจิทัลที่ใช้ในคอมพิวเตอร์ เช่น สเต็ปมิ่งมอเตอร์ต้องใช้แรงดันขนาด 12 โวลต์ (V) ในการขับเคลื่อนแต่ละขด หรืออุปกรณ์เซ็นเซอร์ (Sensor) ต่างๆ ที่เป็น dry contact switch จะต้องใช้แรงดัน 5 โวลต์ ในการ

สร้างสัญญาณที่ที่แอลเพื่อป้อนให้กับอินพุตของโปรแกรม ดังนั้นวงจรส่วนนี้จึงทำหน้าที่เหมือนเป็นตัวปรับแรงดันของสัญญาณอินพุตนั่นเอง

3.3.4.1 คุณสมบัติของเกรนอินเตอร์เฟส

- รับส่งข้อมูลระหว่างเครื่องคอมพิวเตอร์ผ่านทางพอร์ตขนาน
- ใช้พอร์ตข้อมูล (Data port) 8 บิตของพอร์ตขนานในการรับและส่งข้อมูล (data)
- การรับอินพุตและเอาต์พุตสามารถทำได้โดยส่งคำสั่งควบคุมมายังพอร์ตควบคุม (Control port) ของพอร์ตขนาน
- สามารถสร้างสัญญาณอินเตอร์รัพ (Interrupt signal) เป็นเวลาสั้นๆ (ประมาณ 100-200 ns) เพื่อกระตุ้นให้โปรแกรมทำงานช่วงสั้นๆ
- เมื่อเกิดอินเตอร์รัพข้อมูลอินพุตจะต้องปรากฏบนพอร์ตข้อมูลอย่างต่อเนื่องหรือจนกระทั่งโปรแกรมอ่านค่าเรียบร้อยแล้ว
- สามารถส่งข้อมูลเอาต์พุตให้กับสเต็ปมิ่งมอเตอร์ ได้โดยตรงและต่อเนื่อง
- ต้องมีหัวเสียบสำหรับรับอินพุตจากชุดเกรน

3.3.4.2 crane interface block diagram



รูปที่ 3-11 แสดง Block diagram ของวงจรเกรนอินเตอร์เฟส

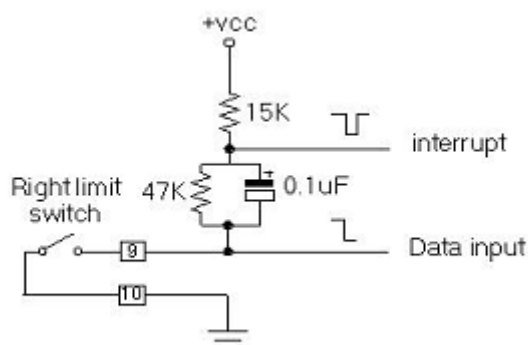
3.3.4.3 การออกแบบวงจร

จากบล็อกไดอะแกรมสามารถนำมาเขียนวงจรตามคุณสมบัติที่ต้องการได้ดังนี้

● วงจรรับสัญญาณ dry contact

สิ่งที่ทราบว่าสวิตช์ (switch) หรืออุปกรณ์ที่ไม่มีไฟเลี้ยงอื่นๆ ต้องการแรงดันในการทำงาน ถ้าเป็นการใช้งาน input data ทั่วๆไปอาจใช้ตัวต้านทานเพียงตัวเดียวในการทำงาน แต่เนื่องจากผู้จัดทำต้องการสร้างพัลส์ (pulse) เพื่อใช้ในการกระตุ้นสัญญาณอินเตอร์รัพจึงนำคาปาซิเตอร์ (Capacitor) มาต่อเพิ่มเติมเพื่ออาศัยช่วงเวลาเวลาที่คาปาซิเตอร์กำลังชาร์จประจุสร้างพัลส์ขึ้นและเมื่อคาปาซิเตอร์ชาร์จประจุจนเต็มสัญญาณอินเตอร์รัพจะหยุดโดยอัตโนมัติ

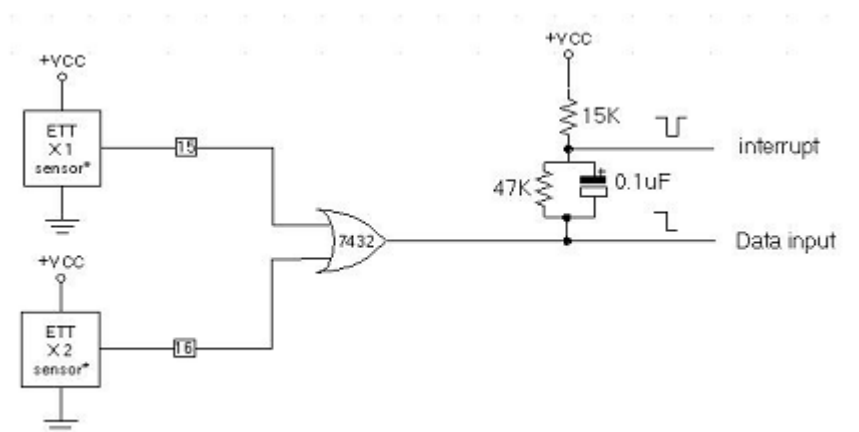
ส่วนหน้าที่ของรีซิสเตอร์ 47K คือใช้ในการคายประจุ (discharge) ประจุออกจากคาปาซิเตอร์กรณีที่สวิตช์ถูกอยู่ในสภาวะออฟ (OFF) แล้ว



รูปที่ 3-12 แสดงวงจรรับสัญญาณ dry contact จากอุปกรณ์ภายนอก

- วงจรรับสัญญาณจากอุปกรณ์เซ็นเซอร์

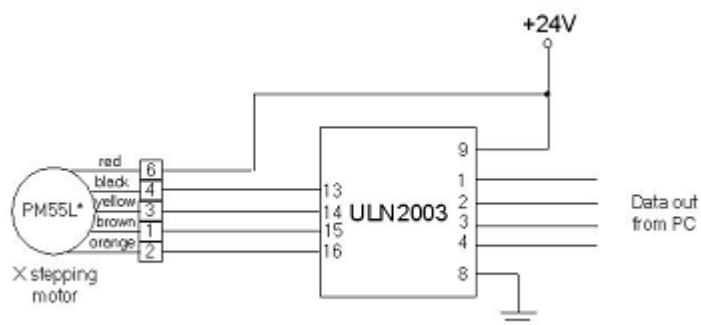
เนื่องจากอุปกรณ์เซ็นเซอร์ต่างๆ ส่วนใหญ่ใช้วงจรออฟแอมป์ (OP AMP) เป็นตัวตรวจวัดความแตกต่าง จึงสามารถที่จะจ่ายแรงดันให้กับคอมพิวเตอร์ได้โดยตรง แต่จะใช้หลักการเดียวกับวงจรรับสัญญาณ dry contact เพื่อสร้างสัญญาณ interrupt ดังรูป



รูปที่ 3-13 แสดงวงจรรับสัญญาณจากอุปกรณ์เซ็นเซอร์ภายนอก

- วงจรขับสเต็ปมอเตอร์

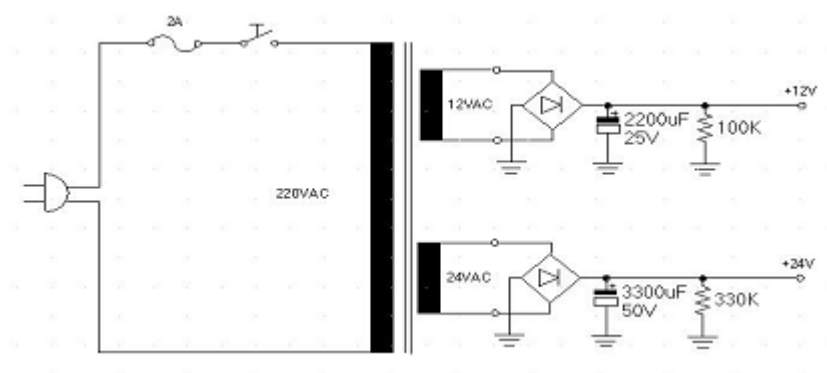
เนื่องจากแรงดันที่จ่ายออกมาจากเครื่องคอมพิวเตอร์เป็นแรงดันระดับต่ำประมาณ 5 โวลต์ การที่จะนำไปใช้ขับสเต็ปมอเตอร์ได้จึงต้องมีการปรับแรงดันให้เหมาะสมกับความต้องการของสเต็ปมอเตอร์ โดยใช้ไอซี (IC) ULN2003 เป็นตัวจัดการเรื่องนี้โดยสามารถคุณสมบัตินี้ได้จากภาคผนวก ง



รูปที่ 3-14 แสดงวงจรการปรับระดับสัญญาณที่ส่งไปยังสเต็ปปีงมอเตอร์

- วงจรจ่ายแรงดัน

เนื่องจากการทดสอบการทำงานของสเต็ปปีงมอเตอร์ ได้มีการทดสอบกับแรงดันทั้ง 24 โวลต์ และ 12 โวลต์ เพื่อทดสอบแรงบิดของมอเตอร์ (motor) และตรวจสอบปริมาณความร้อนที่สะสมบนตัวมอเตอร์ ดังนั้นจึงต้องออกแบบแรงดันขับเคลื่อนมอเตอร์ไว้ 2 ระดับดังรูป



รูปที่ 3-15 แสดงวงจรเพาเวอร์ซัพพลายเพื่อจ่ายไฟให้กับสเต็ปปีงมอเตอร์

หมายเหตุ วงจรที่สมบูรณ์สามารถดูได้จากภาคผนวก จ

3.3.5 การออกแบบเคาน์เตอร์และโครง

เคาน์เตอร์และโครงเป็นส่วนโครงสร้างการทำงานทางกลของเคาน์เตอร์ทั้งหมด โครงสร้างทั้งหมดถูกสร้างขึ้นจากอะลูมิเนียม โดยเคาน์เตอร์และโครงที่ต้องการมีคุณสมบัติดังต่อไปนี้

3.3.5.1 คุณสมบัติของเคาน์เตอร์และโครง

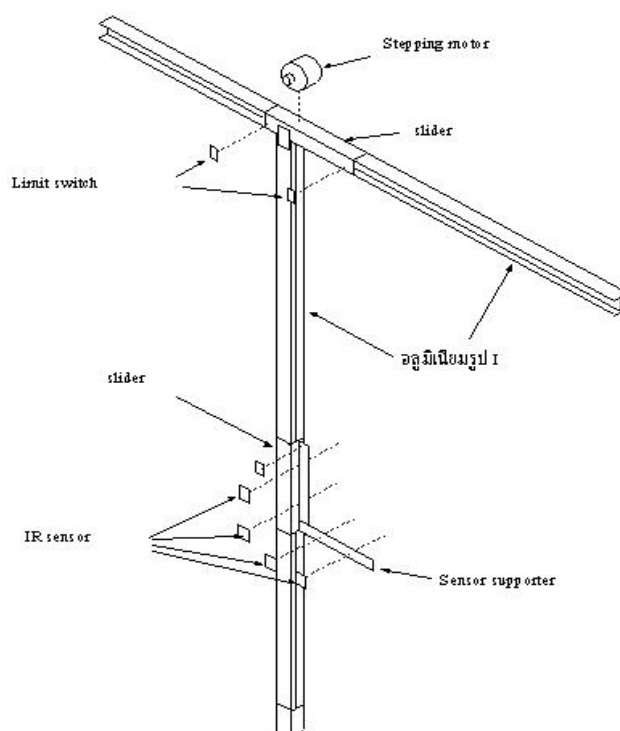
- ตัวเคาน์เตอร์สามารถเคลื่อนไหวได้ทั้งแนวตั้งและแนวนอนด้วยสเต็ปปีงมอเตอร์จำนวน 2 ตัว
- โครงมีขนาดประมาณ 80 cm. x 80 cm. มีจำนวนช่องเท่ากับ 5x5 ช่องแต่ละช่องมีขนาด 16 cm. x 18 cm. ขอบของช่องแต่ละช่องมีความกว้างประมาณ 1 cm. เพื่อใช้ในการตรวจจับของ

เซ็นเซอร์และโครงสร้างของโครงสามารถวางได้อย่างแข็งแรงและมั่นคง

- มีลิมิตสวิตช์ (limit switch) สำหรับตรวจจับการเคลื่อนที่ของเครนทั้ง 4 ด้านคือ ซ้าย, ขวา, บน และล่าง
- มีตัวตรวจจับตำแหน่งเครน 4 ตัว ติดตั้งที่เครนทั้ง 2 แนวโดยแต่ละแนวจะใช้ตัวเซ็นเซอร์แนวละ 2 ตัว เพื่อตรวจสอบความถูกต้องของสัญญาณที่ส่งให้กับ interface card
- ต้องมีการสร้างคอนเน็คเตอร์ (connector) สำหรับเชื่อมต่อขาสัญญาณอินพุตและเอาต์พุตทั้งหมดไปยังส่วน crane interface card

3.3.5.2 การออกแบบโครงสร้างของเครน

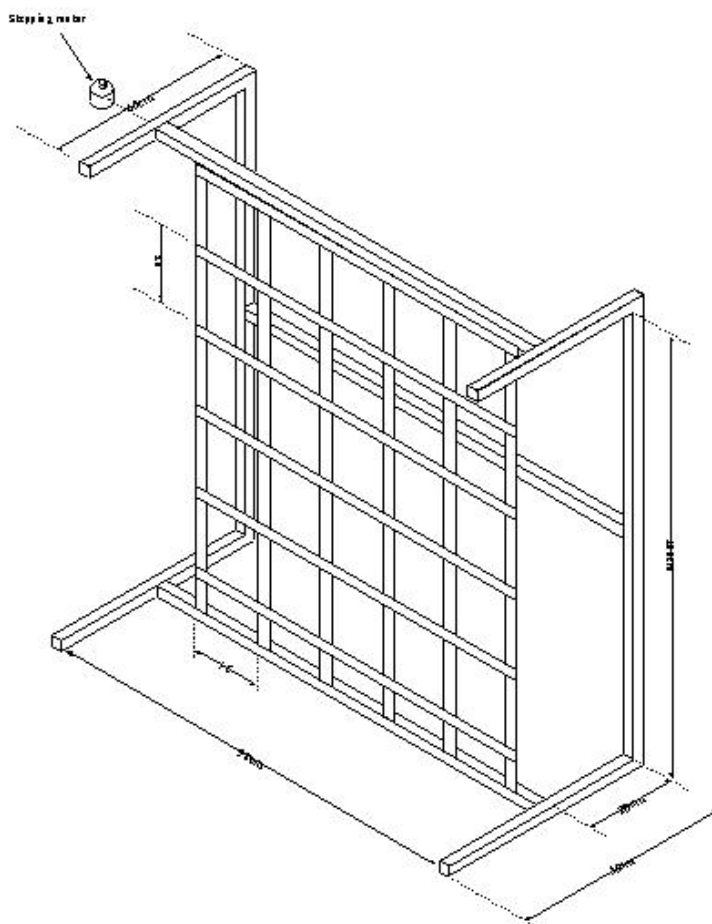
ในการออกแบบโครงสร้างของเครน ไม่มีรูปแบบในการสร้างที่แน่นอนขึ้นอยู่กับความสะดวกในการจัดหาวัสดุ และการเลือกโครงสร้างที่ไม่ซับซ้อนที่สามารถเคลื่อนไหวได้ง่ายและคงที่ จากการศึกษาการทำงานของระบบเครนประเภทต่างๆ ได้ข้อสรุปว่าเครนควรมีลักษณะคล้ายแขนของเครื่องพล็อตเตอร์ (plotter) โดยส่วนที่เคลื่อนที่จะอยู่ในแนวตั้งเพื่อลดปัญหาเนื่องจากแรงโน้มถ่วง



รูปที่ 3-16 แสดงลักษณะโครงสร้างของเครน

3.3.5.3 การออกแบบโครงสร้างของโครง

ปัญหาของการออกแบบโครงสร้างของโครงอยู่ที่ทำอย่างไรให้โครงแข็งแรงและมั่นคง โดยใช้โครงสร้างที่ซับซ้อนน้อยที่สุด จากการทดลองและทดสอบโครงสร้างหลายรูปแบบ ในที่สุดผู้จัดทำได้ข้อสรุปเกี่ยวกับรูปแบบของโครงสร้างที่มีความแข็งแรงและไม่ซับซ้อน ดังนี้



รูปที่ 3-17 แสดงลักษณะโครงสร้างของโครง

3.4 การติดตั้งและปรับแต่งเซิร์ฟเวอร์

3.4.1 การติดตั้ง Debian Linux

Debian linux v.3.0 r2 เป็นแผ่นติดตั้งที่สามารถเลือกได้ว่าจะติดตั้งด้วย kernel 2.2 หรือ kernel 2.4 โดยในการทดลองนี้เลือกที่จะพัฒนาด้วย kernel 2.4 ด้วยเหตุผลที่ว่ามีความทันสมัยและใช้งานง่ายกว่า kernel 2.2 ถึงแม้ว่าในขณะที่ทำการทดลองนี้จะมี kernel 2.6 ออกสู่ตลาดแล้วก็ตาม แต่ด้วยเหตุที่มีเอกสารสนับสนุนน้อยจึงเลือกที่จะทำงานกับ kernel 2.4 ต่อไป

การติดตั้ง Debian Linux ด้วย kernel 2.4 สามารถทำได้โดยพิมพ์คำสั่ง bf24 หลังจากบูตเครื่องคอมพิวเตอร์ด้วยแผ่นซีดีแล้วและในส่วนรายละเอียดของขั้นตอนการติดตั้งสามารถดาวน์โหลดได้ที่ <http://www.debian.org/releases/stable/i386/install.en.pdf>

3.4.2 การปรับแต่งเซิร์ฟเวอร์

การปรับแต่งเซิร์ฟเวอร์หลังการติดตั้งเป็นสิ่งจำเป็นเช่นเดียวกับการติดตั้งระบบ เพราะการเขียนโปรแกรมหรือการทดสอบการทำงานผ่านเว็บเพจอาจจะทำไม่ได้ถ้าไม่ตรวจสอบการทำงานของเซิร์ฟเวอร์ให้สมบูรณ์ โดยจุดที่ต้องตรวจสอบมีดังนี้

3.4.2.1 การตรวจสอบเวอร์ชันของ linux kernel และ linux include file

เนื่องจากการพัฒนาโปรแกรมบนลินุกซ์ในระหว่างรันโปรแกรม ระบบปฏิบัติการจะตรวจสอบเวอร์ชันของโปรแกรมที่กำลังทำงานด้วยว่าเป็นเวอร์ชันเดียวกับเคอร์เนลหรือไม่ ดังนั้นการตรวจสอบเวอร์ชันของเคอร์เนลและ include file จึงเป็นสิ่งจำเป็นสามารถตรวจสอบได้โดย

- ตรวจสอบเวอร์ชันของ linux kernel โดยพิมพ์คำสั่ง

```
uname -a
```

```
Linux project 2.4.18 #2 SMP Thu Sep 30 07:15:47 ICT 2004 i586 unknown
```

- ตรวจสอบเวอร์ชันของ include file โดยพิมพ์คำสั่ง

```
cat /usr/include/linux/version.h
```

```
#define UTS_RELEASE "2.4.18"
```

```
#define LINUX_VERSION_CODE 132114
```

```
#define KERNEL_VERSION(a,b,c) (((a) << 16) + ((b) << 8) + (c))
```

หมายเหตุ ในส่วนที่ขีดเส้นใต้จะต้องตรงกันทั้ง 2 จุดถ้าไม่ตรงกันให้ทำการแก้ไขค่าในไฟล์ /usr/include/linux/version.h ให้ตรงกับเวอร์ชันของเคอร์เนลที่ใช้อยู่

3.4.2.2 การตรวจสอบการทำงานของเน็ตเวิร์กการ์ด

ในเบื้องต้นจะต้องตรวจสอบว่าเน็ตเวิร์กการ์ดทำงานอยู่และ กำหนดหมายเลขไอพีอย่างถูกต้องหรือไม่ ซึ่งหมายเลขไอพีนั้นผู้ใช้งานสามารถเปลี่ยนแปลงและกำหนดใหม่ได้ แต่ควรสอดคล้องกับ Client ที่ใช้ในการพัฒนาโปรแกรม ตามตัวอย่างได้มีการกำหนดหมายเลขไอพีไว้ที่ 192.168.0.50 คำสั่งที่ใช้ตรวจสอบหมายเลขไอพีคือ ifconfig หน้าจอจะแสดงรายชื่อของเน็ตเวิร์กการ์ดที่กำลังทำงานอยู่ทั้งหมด

```
eth0  Link encap:Ethernet HWaddr 00:50:04:BA:99:34
      inet addr:192.168.0.50 Bcast:192.168.0.255 Mask:255.255.255.0
      UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
      RX packets:961 errors:0 dropped:0 overruns:0 frame:0
      TX packets:619 errors:0 dropped:0 overruns:0 carrier:0
      collisions:0 txqueuelen:100
      RX bytes:59476 (58.0 KiB) TX bytes:58146 (56.7 KiB)
      Interrupt:11 Base address:0xe000

lo    Link encap:Local Loopback
      inet addr:127.0.0.1 Mask:255.0.0.0
      UP LOOPBACK RUNNING MTU:16436 Metric:1
      RX packets:83 errors:0 dropped:0 overruns:0 frame:0
```

```
TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:5815 (5.6 KiB) TX bytes:5815 (5.6 KiB)
```

ผู้ใช้ควรให้ความสนใจเฉพาะเน็ตเวิร์กการ์ด eth0 เพราะ lo เป็นโลคอลอินเทอร์เฟซ (local interface) ของระบบ ถ้าปรากฏข้อความดังตัวอย่างด้านบนแสดงว่าเน็ตเวิร์กการ์ดถูกติดตั้งและทำงานอย่างสมบูรณ์แล้ว แต่ถ้าไม่เป็นไปตามที่กล่าวมาให้ทำการตรวจสอบค่าภายในไฟล์ `/etc/network/interfaces` ภายในไฟล์ควรประกอบด้วย

```
# /etc/network/interfaces -- configuration file for ifup(8), ifdown(8)

# The loopback interface
auto lo eth0

iface lo inet loopback

iface eth0 inet static
address 192.168.0.50
netmask 255.255.255.0
```

หมายเหตุ ส่วนที่ขีดเส้นใต้เป็นหมายเลขไอพีของเซิร์ฟเวอร์เมื่อแก้ไขเรียบร้อยแล้วให้ทำการจัดเก็บ (save) และพิมพ์คำสั่ง `/etc/init.d/network restart` เพื่อให้ระบบเน็ตเวิร์กทำงานใหม่อีกครั้ง

3.4.2.3 การตรวจสอบการทำงานของเทลเน็ตและเอฟทีพี

Connected to project.

220 project FTP server (Version 6.4/OpenBSD/Linux-ftpd-0.17) ready.

Name (localhost:root):

จะต้องปรากฏสื่ออินพรีม (login prompt) ดังตัวอย่างด้านบนและเช่นเดียวกับเอฟทีพีในการตรวจสอบ telnet service สามารถทำได้เช่นเดียวกัน โดยพิมพ์คำสั่ง `telnet localhost` จะต้องปรากฏข้อความดังนี้

Trying 127.0.0.1...

Connected to project.

Escape character is '^['.

Debian GNU/Linux 3.0 project

project login:

ข้อความด้านบนจะถูกแสดงในกรณีที่เทลเน็ตและเอฟทีพีกำลังทำงานอยู่ถ้าไม่เป็นไปตามตัวอย่าง ให้ทำการตรวจสอบไฟล์ `/etc/inetd.conf` ว่าทำการเปิดให้บริการเทลเน็ตและเอฟทีพีหรือยัง โดยภายใน `inetd.conf` ควรจะเป็นดังนี้

```
# /etc/inetd.conf: see inetd(8) for further informations.
```

```
#:STANDARD: These are standard services.
```

```
ftp      stream  tcp      nowait  root            /usr/sbin/tcpd  /usr/sbin/in.ftpd
telnet   stream  tcp      nowait  telnetd.telnetd /usr/sbin/tcpd  /usr/sbin/in.telnetd
```

สังเกตว่าหน้าบรรทัดที่มีข้อความเทลเน็ตและเอฟทีพีจะต้องไม่มีเครื่องหมาย “#” อยู่ด้านหน้า ถ้ามีให้ทำการลบเครื่องหมาย “#” ออก จากนั้นทำการจัดเก็บและรีสตาร์ทระบบใหม่และทำการตรวจสอบรายละเอียดดังที่กล่าวมาแล้วอีกครั้ง ถ้ายังไม่สามารถใช้งานได้ให้ตรวจสอบว่าเทลเน็ตและเอฟทีพีได้ถูกติดตั้งหรือยังโดยการตรวจสอบแพ็คเกจที่ติดตั้งไว้ดังนี้

```
dpkg -l |grep ftp
```

```
ii      ftp      0.17-9      The FTP client.
ii      ftpd      0.17-13     FTP server
ii      lftp      2.4.9-1     Sophisticated command-line FTP/HTTP client p
```

```
dpkg -l |grep telnet
```

```
ii      telnet    0.17-18     The telnet client.
li      telnetd    0.17-18     The telnet server.
```

ถ้าไม่เป็นไปตามตัวอย่างด้านบนที่กล่าวมาให้ทำการติดตั้งแพ็คเกจโดยถ้าเซิร์ฟเวอร์สามารถเชื่อมต่ออินเทอร์เน็ตได้ให้ทำการพิมพ์คำสั่ง `apt-get install ftp` เพื่อติดตั้งการให้บริการเอฟทีพีและ `apt-get install telnet` เพื่อติดตั้งการให้บริการเทลเน็ตระบบจะทำการดาวน์โหลดและติดตั้งแพ็คเกจโดยอัตโนมัติ

3.4.2.4 การตรวจสอบการทำงานของเว็บเซิร์ฟเวอร์

การทำงานของเว็บเซิร์ฟเวอร์ (Web server) หรือ Apache สามารถตรวจสอบการทำงานได้โดยใช้คำสั่ง `ps ax|grep apache` จะปรากฏข้อความดังนี้

```
198 ?    S    0:00 /usr/sbin/apache
312 ?    S    0:00 [apache]
313 ?    S    0:00 [apache]
314 ?    S    0:00 [apache]
```

```
315 ?    S    0:00 [apache]
316 ?    S    0:00 [apache]
927 pts/0  S    0:00 grep apache
```

ถ้าไม่มีข้อความที่คล้ายคลึงกับตัวอย่างด้านบนให้ทำการตรวจสอบว่า apache ถูกติดตั้งแล้วหรือยัง โดยใช้คำสั่ง `dpkg -l |grep apache` จะปรากฏข้อความดังนี้

```
ii apache                1.3.26-0woody3 Versatile, high-performance HTTP server
ii apache-common         1.3.26-0woody3 Support files for all Apache web servers
```

ถ้าไม่ปรากฏข้อความดังตัวอย่าง ให้ทำการติดตั้ง apache โดยใช้คำสั่ง `apt-get install apache` หรือติดตั้งจากแผ่นซีดี Debian linux แผ่นที่ 1 ด้วยคำสั่ง `dpkg -i` ตามด้วยชื่อไฟล์ apache

การคอมไพล์เคอร์เนลเป็นส่วนการปรับแต่งเพิ่มเติมอาจต้องทำเพื่อให้รองรับการทำงานเมนบอร์ดรุ่นอื่นๆ แต่จากการทดสอบพบว่า การนำ Debian Linux kernel 2.4 ไปใช้งานกับเมนบอร์ดตระกูล x86 ไม่จำเป็นต้องคอมไพล์เคอร์เนลใหม่ โดยสามารถใช้งานได้เป็นปกติ หากผู้ใช้ต้องการคอมไพล์เคอร์เนลใหม่สามารถดูรายละเอียดได้จาก www.kernel.org

3.5 การเขียนโปรแกรม

3.5.1 โปรแกรมส่วน Crane controller

ฮาร์ดโค้ดที่สมบูรณ์สามารถดูได้จากภาคผนวก ก. ในส่วนที่จะกล่าวต่อไปเป็นเพียงฮาร์ดโค้ดบางส่วนที่นำมาประกอบคำอธิบายการทำงานและเปรียบเทียบกับโครงสร้างโมดูลที่ทำการออกแบบไว้สามารถแบ่งฮาร์ดโค้ดเป็นส่วนๆ ได้ดังนี้

3.5.1.1 โปรแกรมส่วน Global declaration

```
static struct proc_dir_entry *main_dir,      /* main process directory */
*move_only_file,                            /* move motor only process */
*get_position_file,                         /* read crane position process */
*get_status_file;                          /* read all crane status process */
```

เป็นส่วนที่แสดงโครงสร้างระบบไฟล์ของโปรเซสที่ถูกสร้างขึ้นภายใต้ /proc directory จากโค้ดด้านบนจะแสดงให้เห็นโครงสร้างไคลเร็กทอรีจำนวน 1 ไคลเร็กทอรีประกอบด้วยไฟล์จำนวน 4 ไฟล์

```

/*****

all status messages

*****/

/* crane status */

char *init_string = "Initializing crane please wait\n";
char *standby_string = "Crane standby for your request\n";
char *moving_string = "Crane moving to specified position\n";
char *demo_string = "Program running in demo mode\n";
char *timeout_error_string = "Crane not reach to target position in time\n";
char *left_limit_string = "Left limit action\n";
char *right_limit_string = "Right limit action\n";
char *top_limit_string = "Top limit action\n";
char *bottom_limit_string = "Bottom limit action\n";

/* user warning message */

char *user_position_error = "Please enter X,Y position between 0,0 and 5,5 coordinate\n";
char *user_keyin_error = "Please enter number in format X,Y between 0,0 and 5,5 coordinate\n";

```

บอร์ดโค้ดด้านบนแสดงให้เห็นการกำหนดแมสเสดที่แจ้งให้กับผู้ใช้งานเมื่อระบบอยู่ในสถานะต่างๆ

```

/*****

/* define global variables here */

*****/

int xyposchange=1; /*used for protect x,y position if motor not moving */
int timeout = 0;          /*used for limit motor action */

long xmotorstarttime = 0; /* used for check x sensor interval time */
long ymotorstarttime = 0;

int xflag = 0;          /* start x sensor flag */
int yflag = 0;

int xflag2 = 0;

```



```

int yflag2 = 0;

int xpos=0;           /*crane x position counter */
int ypos=0;           /*crane y position counter */

char xdirect,ydirect;   /* global variable for motor direction */

char progmode = 'a';    /* a = actual , d = demo */

unsigned short motorout ;    /*variable for keep stepping motor data out */

unsigned char ystepable[MAXSTEP+1] = {0x03,0x06,0x0c,0x09};
unsigned char xstepable[MAXSTEP+1] = {0x90,0xc0,0x60,0x30};

int xindex=0;    /*stepper motor move index */
int yindex=0;

```

ส่วนของโปรแกรมด้านบนแสดงให้เห็นตัวแปรโกลบอลทั้งหมดที่ใช้ในการทำงาน

3.5.1.2 โปรแกรมส่วน Module initialize

ซอร์สโค้ดส่วนนี้จะถูกเรียกใช้งานเมื่อโมดูลถูกโหลดเข้าสู่เคอร์เนลการทำงานส่วนใหญ่จะเป็นการเตรียมพื้นที่สำหรับทำงานให้แก่โมดูล เช่นการตรวจสอบค่าหมายเลขอินเตอร์รัพว่าสามารถใช้งานได้หรือไม่ หรือสร้าง Directory Tree ให้กับ Process file ทั้งหมดเป็นต้น

```

static int __init init_crane_controller(void)
{
    int rv = 0;
    int i;

    /* Create the crane_controller /proc entry */
    main_dir = proc_mkdir("crane_controller", NULL);
    if(main_dir == NULL) {
        rv = -ENOMEM;
        goto out;
    }
}

```

ส่วนของโปรแกรมด้านบนเป็นการสร้างไดเรกทอรีหลักชื่อ crane_controller ภายใต้ /proc ไดเรกทอรี

```
main_dir->owner = THIS_MODULE;

/* Create liftacmains and make it readable by all - 0444 */
move_only_file = create_proc_entry("move_only", 0666, main_dir);
if(move_only_file == NULL) {
    rv = -ENOMEM;
    goto no_move_only;
}

move_only_file->read_proc = NULL;
move_only_file->write_proc = &proc_move_only;
move_only_file->owner = THIS_MODULE;
```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่ทำหน้าที่รับข้อมูลจากผู้ใช้โดยไฟล์ที่สร้างขึ้นนี้จะรับค่าอินพุตจากผู้ใช้แล้วจะส่งค่าให้กับโปรแกรมย่อย proc_move_only

```
/*status file */
get_status_file = create_proc_entry("crane_status", 0444, main_dir);
if(get_status_file == NULL) {
    rv = -ENOMEM;
    goto no_get_status;
}

get_status_file->data = init_string;
get_status_file->read_proc = &proc_read_status;
get_status_file->write_proc = NULL;
get_status_file->owner = THIS_MODULE;
```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่ทำหน้าที่ส่งข้อมูลไปยังผู้ใช้โดยไฟล์ที่สร้างขึ้นนี้จะส่งค่า status message ให้กับผู้ใช้โดยจะอ่านค่าจาก proc_read_status

```
crane_interrupt_file = create_proc_entry("crane_interrupt", 0444, main_dir);
if(crane_interrupt_file == NULL)
{
    return -ENOMEM;
}
```

```

crane_interrupt_file->read_proc = NULL;
crane_interrupt_file->write_proc = NULL;
crane_interrupt_file->owner = THIS_MODULE;

/* request interrupt from linux */
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,"crane_interrupt",NULL);
if ( rv ) {
    printk("Can't get interrupt %d\n", INTERRUPT);
    goto no_interrupt;
}

```

ส่วนของโปรแกรมด้านบนเป็นการสร้าง Process file ที่จะทำงานทุกครั้งที่เกิดอินเทอร์รัพจากอุปกรณ์ภายนอกโดยเมื่อเกิดอินเทอร์รัพขึ้นจะมีการเรียกใช้โปรแกรมย่อย crane_interrupt_interrupt และ ยังแสดงให้เห็นถึงวิธีการตรวจสอบหมายเลขอินเทอร์รัพที่ต้องการใช้งาน ว่าสามารถใช้งานได้หรือไม่โดยอินเทอร์รัพที่ใช้ในการทดลองนี้คืออินเทอร์รัพหมายเลข 7 ซึ่งเป็นอินเทอร์รัพของปรีนเตอร์

3.5.1.3 โปรแกรมส่วนควบคุมสแต็ปปีงมอเตอร์

โปรแกรมส่วนนี้เป็นส่วนควบคุมการทำงานของสแต็ปปีงมอเตอร์ทั้งหมด ตั้งแต่การรับข้อมูลตำแหน่งจากผู้ใช้และตรวจสอบความถูกต้องของข้อมูลที่รับเข้ามาไปจนถึงตรวจสอบที่ปลายทางที่เกรนกำลังเคลื่อนที่ เป็นต้น

```

static int proc_move_only(struct file *file, const char *buffer,unsigned long count, void *data)

```

ตัวอย่างของโปรแกรมด้านบนแสดงให้เห็นโครงสร้างของการส่งค่าพารามิเตอร์ในการเรียกใช้งาน Process file ที่กำลังทำงานอยู่ในเคอร์เนล

```

{
    long i;
    int tempx,tempy;
    /*char xdirect,ydirect; */
    /* check data input correct */
    if(count > 3)
    {
        printk("user request position more than 2 digit\n");
    }
}

```

```

        get_status_file->data = user_position_error; //set status text to user mistake
        goto nothing_to_do;
    }
    else if(count < 3)
    {
        printk("user request position less than 2 digit\n");
        get_status_file->data = user_position_error; //set status text to user mistake
        goto nothing_to_do;
    }
    else
        printk("move to x=%c y=%c\n",buffer[0],buffer[1]);

    /*change char to int for compare*/
    temp_x = chartoint(buffer[0]);          /*set x target to temp_x */
    temp_y = chartoint(buffer[1]);          /*set y target to temp_y */

    /* check for valid number */
    if((temp_x== -1) || (temp_y== -1))
    {
        printk("user not keyin number\n");
        get_status_file->data = user_keyin_error;    //set status text to user mistake
    }

```

ส่วนของโปรแกรมด้านบนทำหน้าที่ตรวจสอบความถูกต้องของข้อมูลที่ผู้ใช้ป้อนให้กับระบบ ถ้าข้อมูลที่ป้อนไม่ถูกต้องเช่น ไม่ใช่ตัวเลข หรือป้อนเกินค่าที่กำหนดจะแสดงข้อความผิดพลาด

```

else    /* every thing ok so we move the motor */
{
    printk("x and y motor moving\n");
    get_status_file->data = moving_string ;
    if((temp_x==0)&&(temp_y==0))
        move_to_00();
    else
    {

```

```

timeout =0;
xyposchange = 0;
while(!((temp_x == xpos) && (temp_y == ypos))&&(timeout <
    MAXTIMEOUTDELAY))
    /* wait for x and y position */
{
    /* check x position for move */
    if(temp_x < xpos)
        xdirect = 'r';    /* move reverse direction */
    else if(temp_x > xpos)
        xdirect = 'f';    /* move forward direction */
    else
        xdirect = 'n';    /* not move */
    /* check y position for move */
    if(temp_y < ypos)
        ydirect = 'r';    /* move reverse direction */
    else if(temp_y > ypos)
        ydirect = 'f';    /* move forward direction */
    else
        ydirect = 'n';    /* not move */

    move_x_motor();
    move_y_motor();
    send_outport();
    mydelay(MAXMOTORDELAY);
    timeout ++;
}

printf("crane stand by \n");
get_status_file->data = standby_string ;
}
}

if(timeout >= MAXTIMEOUTDELAY)
{
    get_status_file->data = timeout_error_string ;

```

```

        move_to_00();
    }

```

เมื่อข้อมูลที่ป้อนเข้ามาถูกต้อง จากนั้นโปรแกรมจะทำการหมุนมอเตอร์ โดยจะมีจุดสิ้นสุดเมื่อค่าของตำแหน่งเครนเท่ากับตำแหน่งที่ต้องการ แต่ถ้าเครนไม่สามารถไปยังจุดที่กำหนดได้ทันเวลาจะมีการสั่งให้เครนเคลื่อนไปยังตำแหน่ง 0,0

3.5.1.4 โปรแกรมส่วนการทำงานเมื่อเกิดอินเทอร์รัพ

เป็นส่วนของโปรแกรมที่จะถูกเรียกใช้งานเมื่อเกิด หมายเลขอินเทอร์รัพตามที่ได้ลงทะเบียนไว้ ขณะทำเริ่มต้นระบบ โดยมีหน้าที่ในการควบคุมการตอบสนองต่อการเกิดอินเทอร์รัพของโปรแกรม

```

void crane_interrupt_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    CheckInputAndAction();
}

```

ตัวอย่างของโปรแกรมด้านบนแสดงให้เห็น โครงสร้างการส่งค่าพารามิเตอร์ที่เกิดขึ้นขณะที่โปรเซสถูกเรียกผ่านอินเทอร์รัพโปรเซสที่เกิดขึ้นในเคอร์เนลและได้มีการเรียกใช้โปรแกรมย่อยอีกครั้งหนึ่ง

```

void CheckInputAndAction()
{
    int ipA;
    outb(SPPPORTREAD|INPUTENABLE,SPPCONTROLPORT);
    ipA = inb(SPPDATAPORT);
    if(ipA & 0x80)                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
    if(ipA & 0x20)                /* top limit switch */
    {
        ydirect = 'n';
        ypos = 6;
    }
}

```

```

        if(ipA & 0x40)                /*left limit switch */
        {
            xdirect = 'n';
            xpos = 0;
        }
        if(ipA & 0x10)                /* right limit switch */
        {
            xdirect = 'n';
            xpos = 6;
        }
        if(ipA & 0x01)                /* x sensor */
        {
            /* x sensor action see details in appendix */
        }
        if(ipA & 0x02)                /* y sensor */
        {
            /* y sensor action see details in appendix */
        }
        outb(SPPINTERRUPTENABLE|OUTPUTENABLE|OUTPUTLATCH,SPPCONTROLPORT);
    }

```

โปรแกรมย่อยด้านบนจะถูกเรียกใช้งานเมื่อเกิดอินเตอร์รัพ โดยโปรแกรมจะอ่านค่าอินพุตจากพอร์ตนานเพื่อหาที่มาของสัญญาณอินเตอร์รัพ เมื่อหาจนพบจากนั้นโปรแกรมจะทำงานตามโปรแกรมย่อยที่กำหนดต่อไป

3.5.1.5 โปรแกรมส่วนแสดงข้อมูลให้กับผู้ใช้

ดังที่กล่าวไว้แล้วว่าข้อมูลที่โปรแกรมส่งให้กับผู้ใช้มีเพียงสถานะและตำแหน่ง ตัวอย่างซอร์สโค้ดด้านล่างเป็นโครงสร้างของโปรแกรมที่ทำหน้าที่ส่งค่ากลับไปให้กับผู้ใช้งาน

```
/*proc_read_position
```

```
* this process used for send crane position to user when user read position_file
```

```
*/
```

```
static int proc_read_position(char *page,char **start, off_t off, int count, int *eof, void *data)
```

```

{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}

/*proc_read_status
 * this process used for send crane status to user when user read status_file
 */

static int proc_read_status(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    char *outchar = (char *)data;
    int i=0;
    while((outchar[i] != '\n') && (i<255))
    {
        page[i] = outchar[i];
        i++;
    }
    page[i] = '\n';
    i++;
    return i;
}

```

3.5.2 โปรแกรมติดต่อกับผู้ใช้ผ่านเว็บเพจ

โปรแกรมนี้จะถูกเก็บไว้ในไดเรกทอรีของเว็บเซิร์ฟเวอร์ที่รองรับการทำงาน CGI Script โดยตำแหน่งไดเรกทอรีบน debian linux นั้นจะอยู่ที่ /usr/lib/cgi-bin/

```
#!/usr/bin/perl
```

```
#####
```

```
# crane web interface script\
```

```
#
```



```
# created by sthaporn
# 10/10/47      rev.2

#####

# read data from user request
read(STDIN, $dat, $ENV{'CONTENT_LENGTH'});
```

ด้านบนเป็นส่วนของโปรแกรมที่ทำหน้าที่รับค่าอินพุตที่ผู้ใช้งานส่งมายังสคริปต์

```
if(length($dat)==0)
{
    $xpos=" ";
    $ypos=" ";
}
else
{
    ($temp1,$temp2) = split(/&/,$dat);
    ($temp11,$temp12) = split(/=/,$temp1);
    ($xpos,$ypos) = split(/\%2C/,$temp12);
    `echo $xpos$ypos > /proc/crane_controller/move_only`;
}
```

จากนั้นโปรแกรมจะทำการตรวจสอบค่าอินพุตที่รับเข้ามาว่ามีหรือไม่

```
# set base color
$white="FFFFFF";
$red="FF0000";
@color = [];
# reset all cells color to white
$color[11]=$white;

#####

# reset all cell to white color

#####

$color[55]=$white;
```

จากนั้นโปรแกรมจะทำการสร้างอาร์ตเท่ากับจำนวนช่องของโครงและทำการตั้งค่าทุกช่องให้เป็นสีขาวทั้งหมด

```
#####
```

```
# read condition and set display here
```

```
#####
```

```
$status = `cat /proc/crane_controller/crane_status`;
```

```
($status_no,$status_text)=split(/\./,$status);
```

```
$xypos = `cat /proc/crane_controller/crane_position`;
```

```
$color[$xypos] = $red;
```

จากนั้น โปรแกรมจะทำการอ่านค่าตำแหน่งและสถานะจาก Process file ทั้ง crane_status และ crane_position จากนั้นจะนำค่าที่อ่านได้เก็บไว้ในตัวแปร \$xypos เพื่อใช้ชี้ตารางที่ต้องการระบายแดงในบรรทัดสุดท้าย

```
#####
```

```
# display html page script
```

```
#####
```

```
print "Content-type: text/html\n\n";
```

```
print "<html>\n";
```

```
if($status_no==5)
```

```
{
```

```
    print "<meta http-equiv='refresh' content='1;url=http://192.168.0.50/cgi-bin/main.cgi'>\n";
```

```
}
```

```
    print "<title>Crane controller project\n";
```

```
    print "</title>\n";
```

```
    print "<body>Rack Display\n";
```

```
    print "<table border='1' width='25%'>\n";
```

```
    print "<tr>\n";
```

```
    print "<td width='20%' align='center' bgcolor='\"$color[15]\">15</td>\n";
```

```
#####
```

```
# draw rack table
```

```
#####
```

```
    print "<td width='20%' align='center' bgcolor='\"$color[51]\">51</td>\n";
```

```

print "</tr>\n";

print "</table>\n";
print "<br>$status_text\n";
print "<br><form method=\"POST\" action=\"./main.cgi\">\n";
print "<br>Please fill X and Y position in format X,Y<br>\n";
print "<br><input type=\"text\" name=\"xyposition\"><br>\n";
print "<br><input type=\"submit\" value=\"Start move\" name=\"moveb\">\n";
print "<input type=\"reset\" value=\"Clear\" name=\"resetb\">\n";
print "</form>\n";
print "</body>\n";
print "</html>\n";

```

ส่วนสุดท้ายของโปรแกรมเป็นการสร้างฟอร์มสำหรับรับค่าตำแหน่งเป้าหมายของเครนจากผู้ใช้ และทำการส่งค่าตำแหน่งนั้นให้กับตัวเองเพื่อให้เริ่มการทำงานอีกครั้งหนึ่ง

3.5.3 ทดสอบการทำงาน

จากการทดสอบการทำงานของโมดูล crane_controller.o ด้วยวิธีการจำลองป้อนค่าอินพุต โดยตรงจากผู้ใช้และป้อนค่าตำแหน่งของเครนด้วยตัวของผู้ใช้เองพบว่าสามารถทำงานได้เป็นอย่างดีโดยโปรแกรมสามารถสร้าง Process file ทั้งหมดได้สำเร็จ และสามารถทำการส่งและรับข้อมูลตอบโต้กับผู้ใช้งานได้เป็นปกติ

ในส่วนโปรแกรมติดต่อกับผู้ใช้ผ่านเว็บเพจ main.cgi ผู้ใช้สามารถเปิดเว็บเบราว์เซอร์โดยเรียกยูอาร์แอลที่ <http://192.168.0.50/cgi-bin/main.cgi> (ขึ้นอยู่กับไอพีแอดเดรสที่ผู้ใช้งานตั้งไว้) จาก Client เครื่องอื่นพบว่าสามารถเปิดใช้งาน และ ส่งค่าที่ผู้ใช้ป้อนไปยัง Process file ได้อย่างถูกต้อง

3.6 การจัดทำเครนอินเตอร์เฟซ

3.6.1 การเตรียมอุปกรณ์

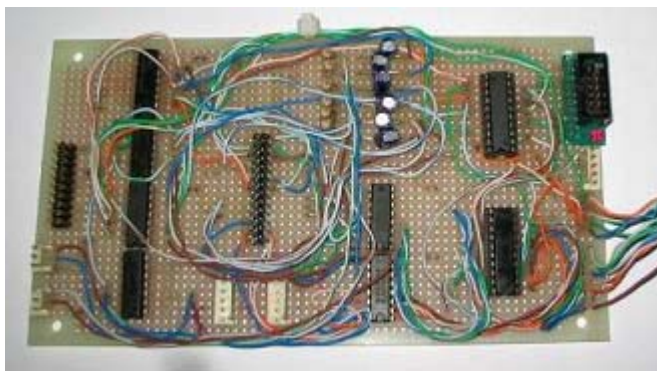
เตรียมอุปกรณ์อิเล็กทรอนิกส์ที่ใช้ในการทำงานดังนี้

แผ่นวงจรเอนกประสงค์	1	แผ่น
IC 74533	1	ตัว
IC 74244	1	ตัว
IC ULN2003	2	ตัว
IC 7408	2	ตัว
IC 7432	1	ตัว

IC 7404	1	ตัว
R 15K	6	ตัว
R 47K	6	ตัว
C 0.1uF	6	ตัว
male socket 25 pin	1	ตัว
male socket ตัวผู้ 2 pin	3	ตัว
parallel port female	1	ตัว
multimeter	1	ตัว

3.6.2 การประกอบวงจรเครื่องอินเตอร์เฟซ

การประกอบวงจรจะทำการประกอบตามวงจรในรูปด้านล่าง ซึ่งวงจรที่ประกอบเสร็จแล้วมีลักษณะดังรูป



รูปที่ 3-18 แสดงแผ่นวงจรเครื่องอินเตอร์เฟซหลังจากประกอบอุปกรณ์เรียบร้อยแล้ว



รูปที่ 3-19 แสดงวงจรเพาเวอร์ซัพพลาย

3.6.3 ทดสอบการทำงาน

เมื่อประกอบแผ่นวงจรเรียบร้อยแล้วได้ทำการตรวจสอบการทำงานของแผ่นวงจรโดยการป้อนสัญญาณอินพุตเข้าไปยังวงจรโดยตรง ทั้งจาก dry contact และเซ็นเซอร์จากนั้นทำการวัดแรงดันที่เอาต์พุตด้วยมัลติมิเตอร์ผลปรากฏว่าแรงดันเอาต์พุตทุกจุดสามารถทำงานได้อย่างเป็นปกติ

เนื่องจากมัลติมิเตอร์มีความไวในการวัดต่ำจึงไม่สามารถวัดค่าความกว้างพัลส์ของสัญญาณอินพุตได้ ดังนั้นการตรวจสอบสัญญาณอินพุตจึงต้องทดสอบกับโปรแกรมใช้งานจริง

3.7 การจัดทำเครนและโครง

3.7.1 การเตรียมอุปกรณ์

อะลูมิเนียมหน้าตัดรูป T ขนาดหน้าตัด กว้าง 2.5cm. X สูง 2.5cm.	2	ท่อน
อะลูมิเนียมหน้าตัดรูป L ขนาดหน้าตัด กว้าง 2.0cm. X สูง 2.0cm.	1	ท่อน
ท่ออะลูมิเนียมสี่เหลี่ยม ขนาดหน้าตัด กว้าง 2.5cm. X ยาว 2.5cm.	1	ท่อน
อะลูมิเนียมเส้นแบน ขนาดหน้าตัด หน้า 2mm. X กว้าง 2cm.	1	เส้น
น็อตเกลียวปล่อยขนาด 3x10	1	กล่อง
น็อตตัวผู้และตัวเมียขนาด 3x10	1	กล่อง
สว่านไฟฟ้า	1	ตัว
เลื่อยเหล็ก	1	อัน

หมายเหตุ อะลูมิเนียมมาตรฐานยาว 6 เมตรต่อท่อน

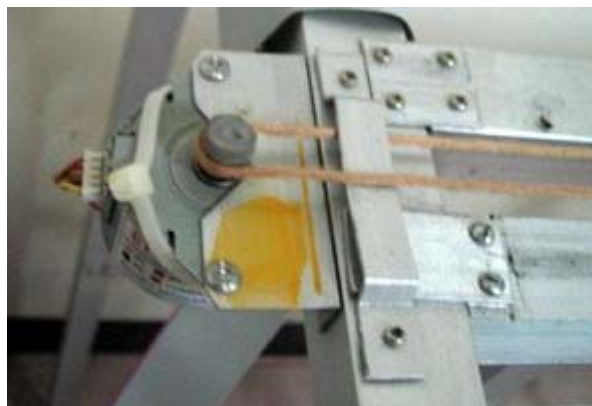
3.7.2 การประกอบอุปกรณ์

3.7.2.1 ส่วนโครงอะลูมิเนียม

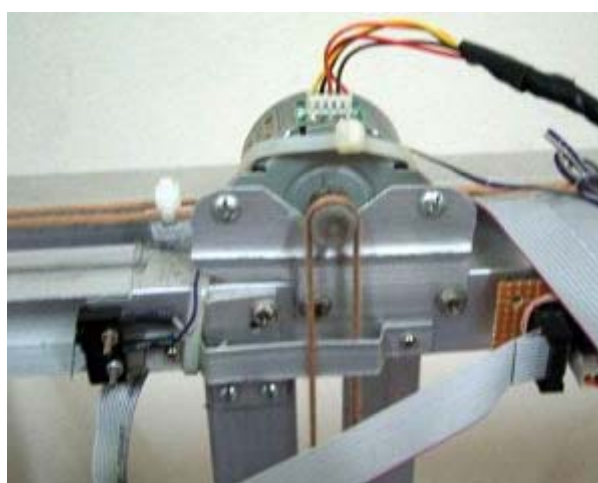
เป็นส่วนของการประกอบโครงอะลูมิเนียม ซึ่งประกอบด้วยการยึดติดแต่ละชิ้นส่วนของอะลูมิเนียมเข้าด้วยกันเพื่อให้ได้ตามที่ออกแบบไว้เพื่อสำหรับใช้ยึดแต่ละส่วนประกอบต่อไป

3.7.2.2 ส่วนติดตั้งสเต็ปมอเตอร์

การติดตั้งสเต็ปมอเตอร์เข้ากับเครนสามารถทำได้โดยทำแผ่นยึดมอเตอร์รูปตัวแอลเจาะรูแผ่นยึดเข้ากับโครงของโครงด้านหนึ่งและอีกด้านยึดติดกับมอเตอร์จากมอเตอร์ทั้ง 2 ตัวมีการใช้เคเบิลไทร์ (cable tie) ยึดด้านหลังไว้เพื่อช่วยต้านแรงดึงจากสายพานที่ทำให้มอเตอร์หมุนไม่คงที่



รูปที่ 3-20 แสดงการติดตั้งสเต็ปป์มอเตอร์ควบคุมแนวนอน



รูปที่ 3-21 แสดงการติดตั้งสเต็ปป์มอเตอร์ควบคุมแนวตั้ง

3.7.2.3 การติดตั้งลิมิตสวิตช์

ในการติดตั้งลิมิตสวิตช์จะกระทำเหมือนกันทุกจุดคือยึดติดกับส่วนปลายของส่วนเคลื่อนที่ทั้งแนวตั้งและแนวนอน จะมีเพียงลิมิตสวิตช์ด้านล่างเท่านั้นที่ไม่สามารถยึดติดกับเกรนได้เพราะพื้นที่ในการเจาะรูนั้นน้อยเกินไป



รูปที่ 3-22 แสดงการติดตั้งลิมิตสวิตช์เข้ากับส่วนเคลื่อนที่แนวนอน

3.7.2.4 โพล์รับสายพาน

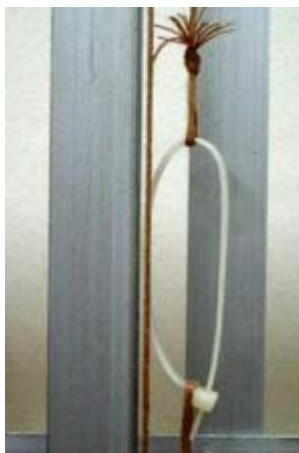
เนื่องจากระบบการหมุนของมอเตอร์ที่ผู้จัดทำออกแบบไว้ให้ใช้มอเตอร์เพียงตัวเดียวในการทำงาน ดังนั้นจึงต้องมีการสร้างโพล์ขึ้นมารองรับด้านตรงข้ามของมอเตอร์ดังรูป



รูปที่ 3-23 แสดงการติดตั้งโพล์ยึดสายพานแนวนอน

3.7.2.5 ส่วนปรับความตึงสายพาน

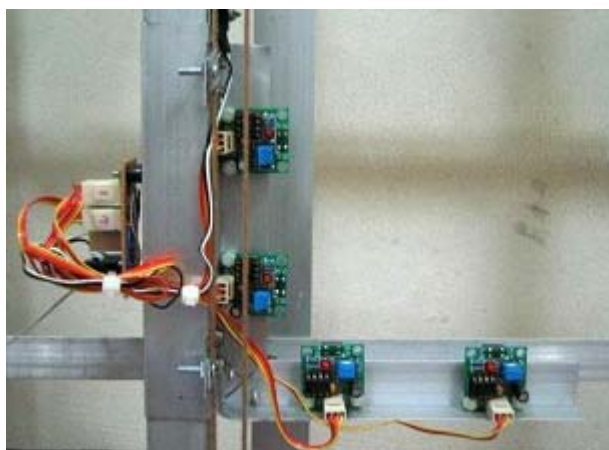
เนื่องจากสายพานที่นำมาใช้งานกับเครนได้ทำการประดิษฐ์สายพานขึ้นใช้เอง โดยสร้างจากเส้นด้ายที่เหนียวจำนวน 30 เส้นทำให้สายพานไม่มีความยืดหยุ่นในการทำงาน จึงต้องมีการใช้เคเบิลไทร์เพื่อใช้เป็นตัวปรับความตึงหย่อนของสายพานดังรูป



รูปที่ 3-24 แสดงการใช้เคเบิลไทรในการปรับความตึงของสายพาน

3.7.2.6 การติดตั้งอุปกรณ์เซ็นเซอร์

การติดตั้งอุปกรณ์เซ็นเซอร์เข้ากับเกรนไม่มีความซับซ้อนมากนัก จะมีจุดเดียวที่ต้องระวังคือ ระยะเซ็นเซอร์จะต้องไม่ห่างหรือใกล้กับโครงจนเกินไป เพราะอาจจะทำให้ไม่สามารถตรวจจับขอบของช่องไม่ได้ถ้าไกลเกินไป หรืออาจจะทำให้ตัวเซ็นเซอร์ชูดกับโครงกรณีที่ติดตั้งใกล้เกินไป การติดตั้งเซ็นเซอร์แสดงดังรูป



รูปที่ 3-25 แสดงการติดตั้งเซ็นเซอร์สำหรับตรวจจับการเคลื่อนที่

3.7.2.7 ส่วนประกอบอื่นๆ

ส่วนอื่นๆนอกเหนือจากที่กล่าวมาข้างต้นที่ควรให้ความสนใจได้แก่ น็อตและสกรูต่างๆที่ยึดติดกับส่วนเคลื่อนที่ ซึ่งต้องคำนึงถึงความยาวของน็อตจะต้องไม่ให้ยาวจนเกินไป เพราะอาจจะไปสัมผัสกับรางหรือโครงได้ ซึ่งจะมีผลต่อการเคลื่อนที่ของเกรนอย่างมาก รูปที่สมบูรณ์ของอุปกรณ์ทั้งหมดเมื่อประกอบเข้าด้วยกันแล้วแสดงดังรูป



รูปที่ 3-26 แสดงโครงและโครงที่สมบูรณ์

3.7.3 ทดสอบการทำงาน

การทดสอบการทำงานของเครนสามารถแบ่งเป็นส่วนๆได้ดังนี้

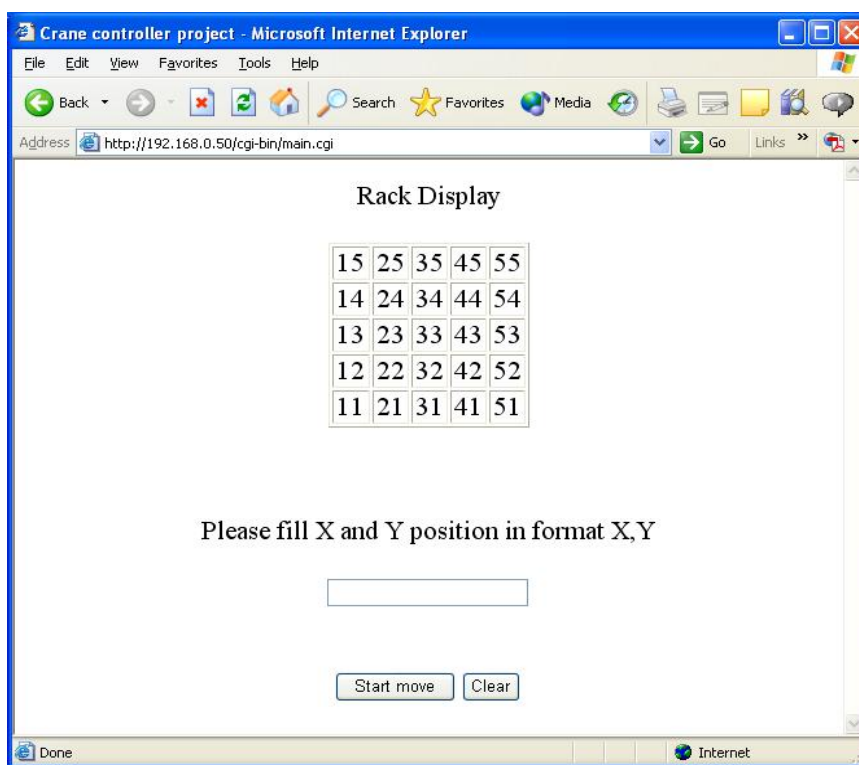
- ทดสอบความมั่นคงแข็งแรงของโครงทำโดยกดบนโครงด้วยแรงพอประมาณ พบว่าสามารถรองรับน้ำหนักได้เป็นปกติ จากนั้นทำการทดสอบโดยการเอียงโครงว่าจะมีผลต่อการเคลื่อนที่ของเครนหรือไม่ พบว่าถ้าโครงเอียงมากจะทำให้เครนเคลื่อนที่ได้ยากไปด้วย
- ทดสอบการเคลื่อนที่ของส่วนเคลื่อนที่ทั้งแนวนอนและแนวตั้งด้วยมือ พบว่ามีความฝืดเล็กน้อยสามารถแก้ไข โดยการตะไบขอบของรางบางส่วนและเติมน้ำมันหล่อลื่นเล็กน้อย
- ทดสอบระยะในการเคลื่อนที่ทุกส่วน เช่น ความห่างระหว่างเครนกับโครง และความห่างของสายแพรกับส่วนเคลื่อนที่ พบว่าระยะห่างเหมาะสมและสามารถทำงานได้เป็นปกติ
- ทดสอบการส่งสัญญาณจากเซ็นเซอร์ทุกตัวมายังอินเตอร์เฟสการ์ดโดยการใช้มัลติมิเตอร์พบว่าสามารถทำงานได้เป็นปกติทุกส่วน

บทที่ 4

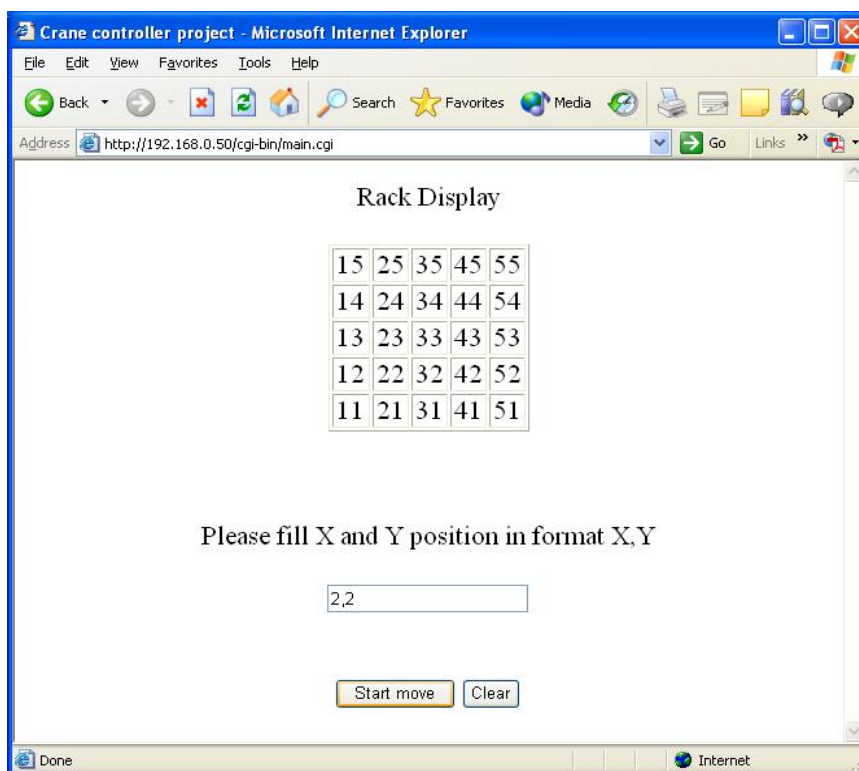
ผลการทำงานและทดสอบระบบ

4.1 ผลการทดสอบระบบ

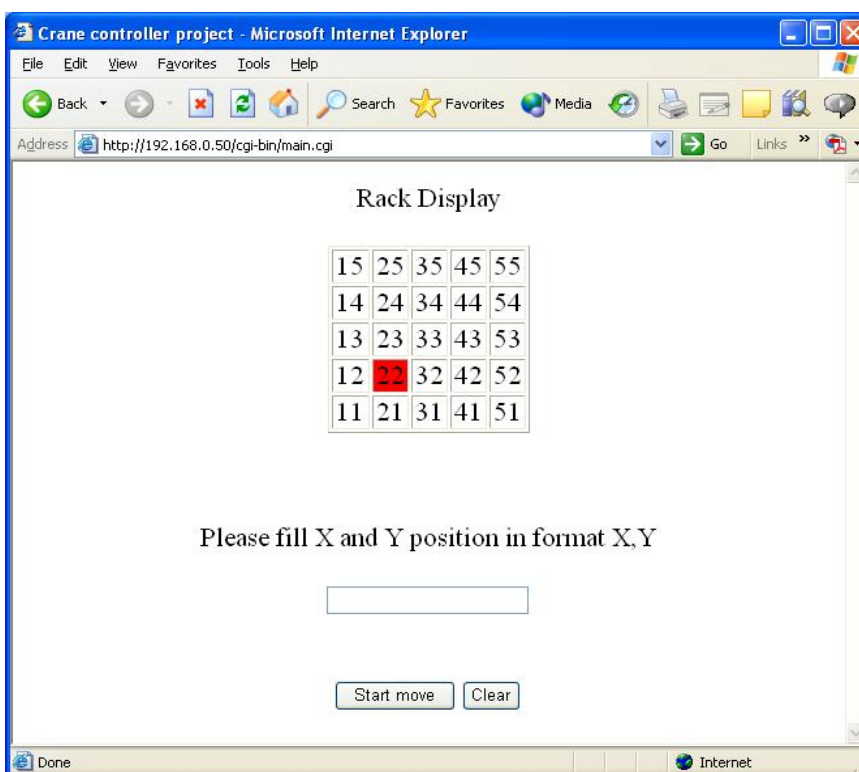
ระบบสามารถทำงานตามความต้องการได้ในระดับหนึ่งกล่าวคือ เมื่อเปิดเซิร์ฟเวอร์ระบบสามารถจัดการเครนให้อยู่ในตำแหน่งเริ่มต้น บริเวณด้านซ้ายล่างสุดของโครงได้ทุกครั้ง และสามารถสั่งเครนให้เคลื่อนที่ไปยังเป้าหมายที่กำหนดโดยผ่านเว็บเพจได้ แต่ยังมีปัญหาในการเคลื่อนที่ของเครนอีกเล็กน้อย กล่าวคือเมื่อเครนเคลื่อนที่จากบริเวณส่วนกลางของโครงไปยังจุดอื่นจะมีการนับตำแหน่งผิดพลาดในบางจุดมีสาเหตุมาจากโครงโครงที่ไม่ขนานเป็นแนวเดียวกับการเคลื่อนที่ของเครนทั้งแนวดิ่งและแนวนอน ส่วนของหน้าเว็บเพจที่ถูกใช้ในการติดต่อกับระบบมีลักษณะดังรูป



รูปที่ 4-1 แสดงส่วนติดต่อกับผู้ใช้เมื่อระบบอยู่ในตำแหน่งเริ่มต้น



รูปที่ 4-2 แสดงส่วนติดต่อกับผู้ใช้เมื่อผู้ใช้กรอกตำแหน่งที่ต้องการให้เครนเคลื่อนที่



รูปที่ 4-3 แสดงส่วนติดต่อกับผู้ใช้เมื่อเครนอยู่ในตำแหน่งเป้าหมาย

4.2 ปัญหาที่พบ

ปัญหาที่พบในระหว่างพัฒนาระบบ สามารถสรุปได้ดังนี้

4.2.1 ปัญหาเกี่ยวกับเซิร์ฟเวอร์

ปัญหาของเซิร์ฟเวอร์ส่วนใหญ่จะอยู่ในช่วงติดตั้งและปรับ และปัญหาที่เกิดขึ้นในขณะทำงานมีเพียงไม่สามารถเรียกใช้งาน CGI Script ที่โหลดขึ้นไปได้ ซึ่งปัญหานี้เป็นปัญหาของโปรแกรม Apache สามารถแก้ไขโดยตรวจสอบสิทธิ์ของไฟล์เดอร์ /usr/lib/script ในไฟล์ /etc/apache/httpd.conf ให้สามารถเรียกใช้งาน CGI Script ได้

4.2.2 ปัญหาเกี่ยวกับโปรแกรม

โปรแกรมเป็นส่วนที่มีปัญหาในการทำงานมากที่สุด แต่ปัญหาที่จะกล่าวต่อไปนี้เป็นเพียงปัญหาบางส่วนที่สำคัญๆ ได้แก่

- ปัญหาเกี่ยวกับวิธีการห้วงเวลาที่ใช้ในโปรแกรม โดยในตอนแรกได้ใช้คำสั่ง `gettimeofday()` ซึ่งจะเป็นการเรียกค่าเวลาหลังจากเที่ยงคืนที่เก็บอยู่ในระบบ โดยวิธีนี้จะมี การรีเซ็ตค่ามิลลิวินาที (millisecond) ทุกๆครั้งที่ครบหนึ่งวินาทีทำให้ไม่สะดวกในการเรียกใช้งาน จากนั้นได้แก้ไขโดยใช้ตัวแปร `jiffies` ซึ่งเป็นตัวแปร tick count มาตรฐานของ linux kernel โดยหลังจากเปิดเครื่องเสร็จแล้วตัวแปร `jiffies` มีค่าเพิ่มขึ้นตลอดเวลาทำให้สามารถใช้เป็นตัวอ้างอิงได้
- ปัญหาเกี่ยวกับการกำเนิดสัญญาณ (oscillate) ของสัญญาณอินพุตโดยเฉพาะตัวเซ็นเซอร์ เมื่อเข้าใกล้หน้าสัมผัสใดๆ จะมีการสร้างพัลส์เข้ามาในโปรแกรมมากมาย ทำให้ต้องใช้วิธีตรวจจับพัลส์แรกที่เข้ามา และใช้การห้วงเวลาเข้ามาประกอบกันเพื่อให้สัญญาณที่ป้อนให้กับโปรแกรมมีความถูกต้องและแม่นยำมากยิ่งขึ้น

4.2.3 ปัญหาเกี่ยวกับคอนโทรลเลอร์เฟส

ปัญหาของอินเตอร์เฟสการ์ดส่วนใหญ่จะเกิดช่วงแรกๆในระหว่างกำลังพัฒนา เช่น การใช้คาปาซิเตอร์ที่มีค่ามากเกินไป ทำให้สัญญาณอินเตอร์รัฟเกิดนานเกินไป เป็นผลให้เมื่อเกิดอินเตอร์รัฟอื่นในช่วงเวลาใกล้เคียงกันจะตรวจไม่พบ หรือการใช้รีซิสเตอร์สำหรับคิซาร์จที่มีค่ามากเกินไปทำให้คาปาซิเตอร์คายประจุได้ช้าทำให้เกิดปัญหาเช่นเดียวกัน

4.2.4 ปัญหาเกี่ยวกับคอนโทรลเลอร์และโครง

- สายพานหลุดจากหัวของสเต็ปริงมอเตอร์ขณะกำลังทำงาน สาเหตุมาจากโพล์ที่ต่อกับหัวของสเต็ปริงมอเตอร์เป็นแบบพื้นเพียงเท่านั้น จึงทำให้ขณะหมุนสายพานจะเลื่อนและหลุดได้ง่าย แก้ไขโดยทำแผ่นอะลูมิเนียมเล็กๆ มายึดติดคร่อมสายพานและนำยางพารามาเคลือบที่บริเวณส่วนโพล์ของมอเตอร์พบว่าสามารถใช้งานได้เป็นอย่างดี

- สายแพรที่เชื่อมระหว่างส่วนเคลื่อนที่แนวนอน เมื่อเครนเคลื่อนที่จะทำให้สายแพรงอเข้ามา ขวางการเคลื่อนที่ของตัวเครนทำให้ไม่สามารถเคลื่อนที่ได้ แก้ไขโดยนำสายวัดในตลับเมตร มาเป็นตัวกันสายแพรไม่ให้เข้ามาขัดกับเครน พบว่าสามารถทำงานได้เป็นอย่างดี
- ปัญหาสตั๊ปปริงมอเตอร์แนวนอนมีกำลังขับไม่พอในการดึงเครน เนื่องจากจะต้องรับน้ำหนักในการลากเครนส่วนที่อยู่แนวตั้งทั้งชิ้น จึงทำให้เกิดการกระตุกและเครนไม่เคลื่อนที่ในแนวนอน สามารถแก้ไขโดยการเพิ่มแรงดันขับให้กับสตั๊ปปริงมอเตอร์เป็น 24 โวลต์ หลังการทดสอบพบว่าสามารถทำงานได้ดี แต่จะเกิดความร้อนสะสมที่มอเตอร์มากขึ้นเล็กน้อย

บทที่ 5

สรุป

5.1 สรุปผลการทำงาน

ถึงแม้การทดลองนี้จะมีปัญหาทางด้านฮาร์ดแวร์บ้าง แต่ฮาร์ดแวร์ไม่ใช่ประเด็นหลักในการศึกษา และทำการทดลองครั้งนี้ ถ้ามองในด้านของซอฟต์แวร์เพียงอย่างเดียวจะเห็นว่าลินุกส์มีความคล่องตัวเป็นอย่างมากในการประยุกต์ใช้งานด้านต่างๆ ไม่ว่าจะเป็นด้านการทำงานกับซอฟต์แวร์เพียงอย่างเดียว หรือในด้านการควบคุมอุปกรณ์ต่างๆ หรือทำงานในลักษณะที่เรียกว่า Embedded Linux นั่นเอง

จากการการทำงานทำให้ทราบถึงขั้นตอนและวิธีการสร้าง Linux Device Driver ตั้งแต่เริ่มจนสามารถใช้งานได้ ถึงแม้ว่าการทดลองนี้จะเป็นการเขียนไดรเวอร์เพื่อควบคุมอุปกรณ์ที่ไม่ซับซ้อนมากนัก แต่ทำให้ได้แนวคิดและวิธีการในการนำ Embedded Linux ไปพัฒนาซอฟต์แวร์ที่มีประสิทธิภาพมากขึ้นในอนาคต

และด้วยเหตุที่ลินุกส์เป็นระบบปฏิบัติการที่สามารถใช้งานได้กับฮาร์ดแวร์ได้หลายแพลตฟอร์มจึงทำให้ลินุกส์จะยังคงเป็นที่ชื่นชอบของนักพัฒนาได้ยาวนานอย่างแน่นอน

5.2 ข้อเสนอแนะ

แนวทางในการทำงานของผู้สนใจทางด้าน Embedded Linux ในอนาคตควรมีเป้าหมายในการนำไปใช้งานอย่างชัดเจนว่าจะนำไปใช้งานด้านใด และควรมีอุปกรณ์นั้นอยู่แล้วจะได้ลดขั้นตอนในการทำงานลงได้ โดยเฉพาะอย่างยิ่งถ้าทดลองกับเครื่องมือหรืออุปกรณ์ที่ไม่เป็นมาตรฐานต่างๆ เช่น กล้องดิจิทัล หรือระบบการทำงานของรถบังคับวิทยุ เป็นต้น จะเป็นประโยชน์สำหรับการทดลองและการนำไปใช้งานในอนาคตเป็นอย่างมาก เพราะในปัจจุบันอุปกรณ์เหล่านี้ได้รับความสนใจมากขึ้นเรื่อยๆ และถ้านำส่วนดีของ Embedded Linux ออกสู่สายตาของผู้ใช้มากขึ้น ในอนาคตจะต้องมีอุปกรณ์ที่ฉลาดมากขึ้นตามไปด้วย

ภาคผนวก ก

โปรแกรมที่สมบูรณ์ของ Crane Controller

```

/*
 * crane_controller program base on liftmon_snowcon program
 * of embedded linux book examples
 * modified for embedded linux project on 5/5/04
 *
 * this program used for control crane simulator set
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/proc_fs.h>
#include <linux/sched.h>
#include <asm/uaccess.h>
#include <linux/ioport.h>
#include <linux/interrupt.h>
#include <asm/io.h>

#define MODULE_VERSION "1.0"
#define MODULE_NAME "crane_controller"

/* All SPP PORT definitions */
#define SPPDATAPORT 0x378
#define SPPSTATUSPORT (SPPDATAPORT + 1)
#define SPPCONTROLPORT (SPPDATAPORT + 2)
#define INTERRUPT 7

/* SPP control port bit definitions */
#define OUTPUTENABLE 0x02
#define OUTPUTLATCH 0x04

```

```

#define INPUTENABLE                0x08
#define SPPPORTREAD                0x20
#define SPPINTERRUPTENABLE        0x10

#define MAXSTEP 3 /*max index of step table */

/* all delay time use in jiffies style */
#define MAXMOTORDELAY 1 /*used for delay motor driving */
#define MAXSENSORDELAY 200 /*used for delay x,y sensor input not occur within */
#define MINSSENSORDELAY 150 /*used for delay x,y sensor input not occur within */
#define MAXINTERRUPTDELAY 1
#define MAXTIMEOUTDELAY 1500

/* SPP input bit definitions */
#define BOTTOMSW                0x01                /* bottom limit switch */
#define TOPSW                  0x02                /* top limit switch */
#define LEFTSW                 0x04                /* left limit switch */
#define RIGHTSW                0x08                /* right limit switch */
#define XPOSSENSOR             0x10                /* crane x position */
#define YPOSSENSOR             0x20                /* crane y position */

/* SPP output bit definitions */
#define XMOTORMASK              0x0F
#define YMOTORMASK              0xF0

MODULE_LICENSE("GPL");

/* These are the pointers to the /proc directory entries */
static struct proc_dir_entry    *main_dir,        /* main process directory */
*move_only_file                /* move motor only process */
*get_position_file,            /* read crane position process */
*get_status_file;              /* read all crane status process */

```

```

/*****

all status messages

*****/

/* crane status */

char *init_string = "Initializing crane please wait\n";
char *standby_string = "Crane standby for your request\n";
char *moving_string = "Crane moving to specified position\n";
char *demo_string = "Program running in demo mode\n";
char *timeout_error_string = "Crane not reach to target position in time\n";
char *left_limit_string = "Left limit action\n";
char *right_limit_string = "Right limit action\n";
char *top_limit_string = "Top limit action\n";
char *bottom_limit_string = "Bottom limit action\n";

/* user warning message */

char *user_position_error = "Please enter X,Y position between 0,0 and 5,5 coordinate\n";
char *user_keyin_error = "Please enter number in format X,Y between 0,0 and 5,5 coordinate\n";

/*****

/* define global variables here */

*****/

int xyposchange=1; /*used for protect x,y position if motor not moving */
int timeout = 0;      /*used for limit motor action */
long xmotorstarttime = 0; /* used for check x sensor interval time */
long ymotorstarttime = 0;
int xflag = 0;        /* start x sensor flag */
int yflag = 0;
int xflag2 = 0;
int yflag2 = 0;
int xpos=0;           /*crane x position counter */
int ypos=0;           /*crane y position counter */

char xdirect,ydirect; /* global variable for motor direction */
char progmode = 'a';  /* a = actual , d = demo */

```

```

unsigned short motorout ;           /*variable for keep stepping motor data out */

unsigned char ystepable[MAXSTEP+1] = {0x03,0x06,0x0c,0x09};
unsigned char xstepable[MAXSTEP+1] = {0x90,0xc0,0x60,0x30};

int xindex=0;                       /*stepper motor move index */
int yindex=0;

/******

/* sub program header define */

/******

void move_to_00(void);
void CheckInputAndAction(void);
void move_x_motor(void);
void move_y_motor(void);
void send_outport(void);
void XPosAction(void);
void YPosAction(void);

/******

/* common use function definind */

/******

void mydelay(long mytimer)
{
    long mydelaytimer;
    mydelaytimer = jiffies;
    while((jiffies - mydelaytimer) < mytimer) ;
}

/* use for change user char input to integer for use in comparison */
int chartoint(int charin)
{
    int retval;
    switch(charin)
    {
        case '0' : retval = 0; break;

```

```

        case '1' : retval = 1; break;
        case '2' : retval = 2; break;
        case '3' : retval = 3; break;
        case '4' : retval = 4; break;
        case '5' : retval = 5; break;
        /*case '6' : retval = 6; break;
        case '7' : retval = 7; break;
        case '8' : retval = 8; break;
        case '9' : retval = 9; break;*/
        default: retval = -1;

    }

    return retval;
}

/* use for move motor in 3 direction */
/* 'r' = reverse direction */
/* 'f' = forward direction */
/* 'n' = none no move */
/* x motor move */
void move_x_motor()
{
    if(xdirect == 'r')
    {
        xindex--;                      /* move x motor backward */
        if(xindex < 0) xindex = MAXSTEP;
    }
    else if(xdirect == 'f')
    {
        xindex++;                      /* move x motor forward */
        if(xindex > MAXSTEP) xindex = 0;
    }
}

```

```

/* y motor move */
/* void move_y_motor(char direction) */
void move_y_motor()
{
    if(ydirect == 'r')
    {
        yindex--;          /* move y motor backward */
        if(yindex < 0) yindex = MAXSTEP;
    }
    else if(ydirect == 'f')
    {
        yindex++;          /* move y motor forward */
        if(yindex > MAXSTEP) yindex = 0;
    }
}

void send_outport()
{
    motorout = 0;
    motorout = ystepable[yindex] | xstepable[xindex];
    outb((motorout & 0xFF) , SPPDATAPORT);
}

/*proc_read_position
* this process used for send crane position to user when user read position_file
*/

static int proc_read_position(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    page[0] = (char) (xpos + 48);
    page[1] = (char) (ypos + 48);
    page[2] = '\n';
    return 3;
}

```

```

/*proc_read_status
* this process used for send crane status to user when user read status_file
*/

static int proc_read_status(char *page,char **start, off_t off, int count, int *eof, void *data)
{
    char *outchar = (char *)data;
    int i=0;
    while((outchar[i] != '\n') && (i<255))
    {
        page[i] = outchar[i];
        i++;
    }
    page[i] = '\n';
    i++;
    return i;
}

/*proc_move_only
* this process used for recieve input x,y position from user and
* move crane to specified position only
*/

static int proc_move_only(struct file *file, const char *buffer,
                          unsigned long count, void *data)
{
    long i;
    int temp_x,temp_y;
    /*char xdirect,ydirect; */
    /* check data input correct */
    if(count > 3)
    {
        printk("user request position more than 2 digit\n");
        get_status_file->data = user_position_error; /* set status text to user mistake */
    }
}

```

```

        goto nothing_to_do;
    }
    else if(count < 3)
    {
        printk("user request position less than 2 digit\n");
        get_status_file->data = user_position_error; /* set status text to user mistake */
        goto nothing_to_do;
    }
    else printk("move to x=%c y=%c\n",buffer[0],buffer[1]);

    /*change char to int for compare*/
    tempx = chartoint(buffer[0]);          /*set x target to tempx */
    tempy = chartoint(buffer[1]);          /*set y target to tempy */

    /* check for valid number */
    if((tempx==-1) || (tempy==-1))
    {
        printk("user not keyin number\n");
        get_status_file->data = user_keyin_error; /* set status text to user mistake */
    }
    else /* every thing ok so we move the motor */
    {
        printk("x and y motor moving\n");
        get_status_file->data = moving_string ;
        if((tempx==0)&&(tempy==0)) move_to_00();
        else
        {
            timeout =0;
            xyposchange = 0;
            while(!((tempx == xpos) && (tempy == ypos))&&(timeout <
                MAXTIMEOUTDELAY))/* wait for x and y position */
            {
                /* check x position for move */
                if(tempx < xpos)

```



```

        xdirect = 'r';      /* move reverse direction */
    else if(tempx > xpos)
        xdirect = 'f';      /* move forward direction */
    else
        xdirect = 'n';      /* not move */

    /* check y position for move */
    if(tempy < ypos)
        ydirect = 'r';      /* move reverse direction */
    else if(tempy > ypos)
        ydirect = 'f';      /* move forward direction */
    else
        ydirect = 'n';      /* not move */

    move_x_motor();
    move_y_motor();
    send_outport();
    mydelay(MAXMOTORDELAY);
    timeout ++;
}
printf("crane stand by \n");
get_status_file->data = standby_string ;
}
}

if(timeout >= MAXTIMEOUTDELAY)
{
    get_status_file->data = timeout_error_string ;
    move_to_00();
}

nothing_to_do:
xyposchange = 1;
return count;
}

```

```

void XPosAction()
{
    if((xyposchange==0))          /* not count if motor not moving */
    {
        if(xdirect == 'r')
            xpos -- ;
        if(xdirect == 'f')
            xpos ++;
    }
}

void YPosAction()
{
    if(xyposchange==0)
    {
        if(ydirect == 'r')
            ypos -- ;
        if(ydirect == 'f')
            ypos ++;
    }
}

void CheckInputAndAction()
{
    int ipA;
    outb(SPPPORTREAD|INPUTENABLE,SPPCONTROLPORT);
    ipA = inb(SPPDATAPORT);
    if(ipA & 0x80)                /*bottom limit switch */
    {
        ydirect = 'n';
        ypos = 0;
    }
}

```

```

if(ipA & 0x20)          /* top limit switch */
{
    ydirect = 'n';
    ypos = 6;
}

if(ipA & 0x40)          /*left limit switch */
{
    xdirect = 'n';
    xpos = 0;
}

if(ipA & 0x10)          /* right limit switch */
{
    xdirect = 'n';
    xpos = 6;
}

if(ipA & 0x01)          /* x sensor */
{
    if(xyposchange==0)
    {
        if(xpos==0)
        {
            if(xflag==0)
            {
                xflag = 1;
                xmotorstarttime = jiffies;
                XPosAction();
                printk("x sensor active  jiffies=%d \n",jiffies);
            }
            else
            {
                if((jiffies-xmotorstarttime) > MAXSENSORDELAY)

```



```

{
    if(ypos==0)
    {
        if(yflag==0)
        {
            yflag = 1;
            ymotorstarttime = jiffies;
            YPosAction();
            printk("y sensor active  jiffies=%d \n",jiffies);
        }
        else
        {
            if((jiffies-ymotorstarttime) > MAXSENSORDELAY)
            yflag = 0;
        }
    }
    else
    {
        if(yflag==0)
        {
            yflag = 1;
            ymotorstarttime = jiffies;
        }
        else
        {
            if(((jiffies-ymotorstarttime) > MINSSENSORDELAY) &&
                (yflag2==0))
            {
                yflag2 = 1;
                YPosAction();
                printk("y sensor active  jiffies=%d \n",jiffies);
            }
        }
    }
}

```

```

        if(((jiffies-ymotorstarttime) > MAXSENSORDELAY) &&
            (yflag2==1))
        {
            yflag2 = 0;
            yflag = 0;
        }
    }
}

outb(SPPINTERRUPTENABLE|OUTPUTENABLE|OUTPUTLATCH,SPPCONTROLPORT);
}

void move_to_00(void)
{
    long i;
    /* initial crane position to -1 */
    xdirect = 'r';
    ydirect = 'r';
    timeout = 0;
    CheckInputAndAction();
    while(((xdirect != 'n') || (ydirect != 'n')) && (timeout < MAXTIMEOUTDELAY) )
    {
        move_x_motor();
        move_y_motor();
        send_outport();
        mydelay(MAXMOTORDELAY);
        timeout ++;
    }
}

```

```

/*function crane_interrupt_interrupt
This function is used for action when interrupt occur */

void crane_interrupt_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    CheckInputAndAction();
}

/* init - init_crane_controller */
static int __init init_crane_controller(void)
{
    int rv = 0;
    int i;

    /* Create the crane_controller /proc entry */

    main_dir = proc_mkdir("crane_controller", NULL);
    if(main_dir == NULL)
    {
        rv = -ENOMEM;
        goto out;
    }

    main_dir->owner = THIS_MODULE;

    /* Create liftacmains and make it readable by all - 0444 */

    move_only_file = create_proc_entry("move_only", 0666, main_dir);
    if(move_only_file == NULL)
    {
        rv = -ENOMEM;
        goto no_move_only;
    }

    move_only_file->read_proc = NULL;
    move_only_file->write_proc = &proc_move_only;
    move_only_file->owner = THIS_MODULE;

```

```

/*status file */

get_status_file = create_proc_entry("crane_status", 0444, main_dir);
if(get_status_file == NULL)
{
    rv = -ENOMEM;
    goto no_get_status;
}

get_status_file->data = init_string;
get_status_file->read_proc = &proc_read_status;
get_status_file->write_proc = NULL;
get_status_file->owner = THIS_MODULE;

/*position file */

get_position_file = create_proc_entry("crane_position", 0444, main_dir);
if(get_position_file == NULL)
{
    rv = -ENOMEM;
    goto no_get_position;
}

get_position_file->read_proc = &proc_read_position;
get_position_file->write_proc = NULL;
get_position_file->owner = THIS_MODULE;

crane_interrupt_file = create_proc_entry("crane_interrupt", 0444, main_dir);
if(crane_interrupt_file == NULL)
{
    return -ENOMEM;
}

crane_interrupt_file->read_proc = NULL;
crane_interrupt_file->write_proc = NULL;
crane_interrupt_file->owner = THIS_MODULE;

```



```

/* request interrupt from linux */
rv = request_irq(INTERRUPT, crane_interrupt_interrupt, 0,"crane_interrupt",NULL);
if ( rv )
{
    printk("Can't get interrupt %d\n", INTERRUPT);
    goto no_interrupt;
}

move_to_00();
if(timeout >= MAXTIMEOUTDELAY)
{
    printk("timeout = %d , xdirect = %c , ydirect = %c \n",timeout,xdirect,ydirect);
    printk("Could not detect crane controller board. Program running in demo mode\n");
    get_status_file->data = demo_string;
}
else
{
    printk("timeout = %d , xdirect = %c , ydirect = %c \n",timeout,xdirect,ydirect);
    printk("Program running in real time mode\n");
    get_status_file->data = standby_string ;
}

/* everything initialed */
printk(KERN_INFO "%s %s initialized\n", MODULE_NAME, MODULE_VERSION);
return 0;

/* this removes /proc entries if we have an error along the way */
no_interrupt:
    remove_proc_entry("crane_interrupt",main_dir);
no_get_position:
    remove_proc_entry("crane_position",main_dir);
no_get_status:
    remove_proc_entry("crane_status",main_dir);

```

```

        no_move_only:

            remove_proc_entry("move_only", main_dir);

        out:

            return rv;
    }

/* exit - cleanup_crane_controller */
static void __exit cleanup_crane_controller(void)
{
    free_irq(INTERRUPT, NULL);

    /* removing the /proc entries */
    remove_proc_entry("crane_interrupt", main_dir);
    remove_proc_entry("crane_position", main_dir);
    remove_proc_entry("crane_status", main_dir);
    remove_proc_entry("move_only", main_dir);
    remove_proc_entry("crane_controller", NULL);

    /* we're done */
    printk(KERN_INFO "%s %s removed\n", MODULE_NAME, MODULE_VERSION);
}

module_init(init_crane_controller);
module_exit(cleanup_crane_controller);

MODULE_AUTHOR("Sthaporn Sahounwong");  MODULE_DESCRIPTION("Crane controller
process module");  EXPORT_NO_SYMBOLS;

```

ภาคผนวก ข

โปรแกรมที่สมบูรณ์ของ main.cgi

```
#!/usr/bin/perl

# crane web interface script\

#

# created by embedded project ,semester 2/2547

# 10/10/47      rev.2

# read data from user request

read(STDIN, $dat, $ENV{'CONTENT_LENGTH'});

if(length($dat)==0)

{

    $xpos=" ";

    $ypos=" ";

}

else

{

    ($temp1,$temp2) = split(/&/,$dat);

    ($temp11,$temp12) = split(/=/,$temp1);

    ($xpos,$ypos) = split(/\%2C/,$temp12);

    `echo $xpos$ypos > /proc/crane_controller/move_only`;

}

# set base color

$white="FFFFFF";

$red="FF0000";

@color = [];

# reset all cells color to white

$color[11]=$white;

$color[12]=$white;

$color[13]=$white;

$color[14]=$white;

$color[15]=$white;
```

```

$color[21]=$white;
$color[22]=$white;
$color[23]=$white;
$color[24]=$white;
$color[25]=$white;

$color[31]=$white;
$color[32]=$white;
$color[33]=$white;
$color[34]=$white;
$color[35]=$white;

$color[41]=$white;
$color[42]=$white;
$color[43]=$white;
$color[44]=$white;
$color[45]=$white;

$color[51]=$white;
$color[52]=$white;
$color[53]=$white;
$color[54]=$white;
$color[55]=$white;

#####
# read condition and set display here
#####

$status = `cat /proc/crane_controller/crane_status`;
($status_no,$status_text)=split(/\./,$status);
$xypos = `cat /proc/crane_controller/crane_position`;
$color[$xypos] = $red;

```

```
#####
# display html page script
#####

print "Content-type: text/html\n\n";
print "<html>\n";
print "<title>Crane controller project\n";
print "</title>\n";
print "<body ><table align=\"center\"><tr><td>Rack Display</td></tr></table><br>\n";
print "<table align=\"center\" border=\"1\" width=\"25%\">\n";
print "<tr>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[15]\">15</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[25]\">25</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[35]\">35</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[45]\">45</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[55]\">55</td>\n";
print "</tr>\n";
print "<tr>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[14]\">14</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[24]\">24</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[34]\">34</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[44]\">44</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[54]\">54</td>\n";
print "</tr>\n";
print "<tr>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[13]\">13</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[23]\">23</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[33]\">33</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[43]\">43</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[53]\">53</td>\n";
print "</tr>\n";
print "<tr>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[12]\">12</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[22]\">22</td>\n";
print "<td width=\"20%\" align=\"center\" bgcolor=\"$color[32]\">32</td>\n";
```

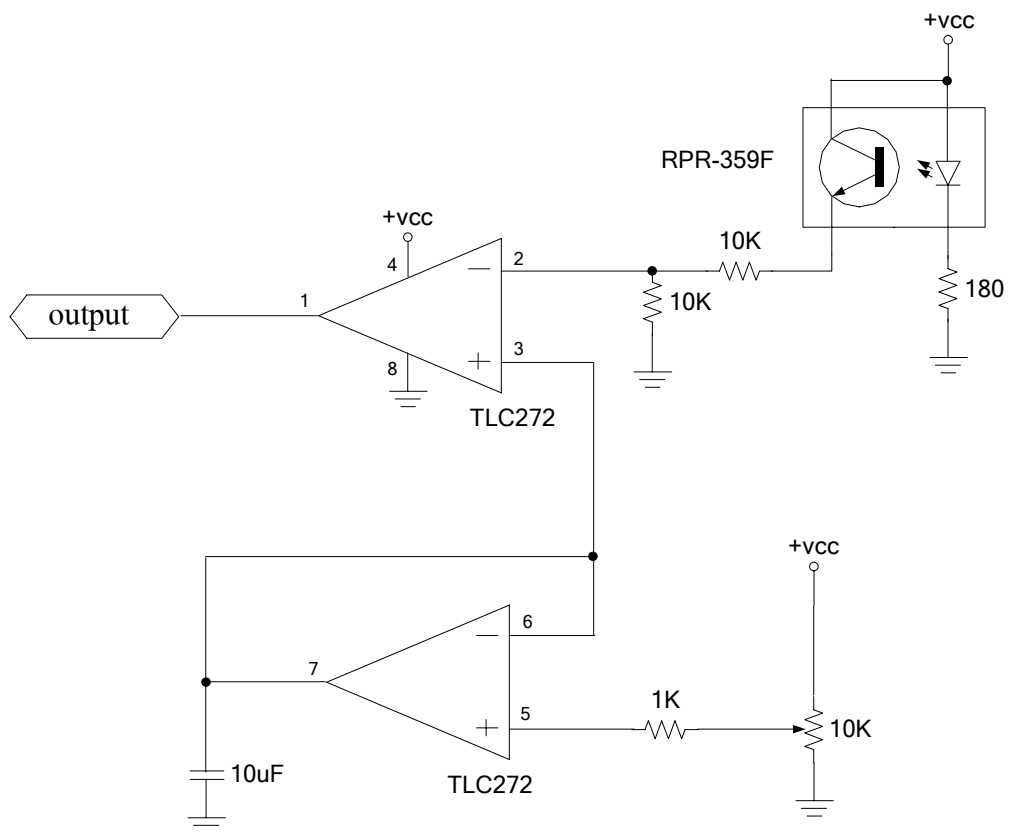
```

print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[42]}\">42</td>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[52]}\">52</td>\\n";
print "</tr>\\n";
print "<tr>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[11]}\">11</td>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[21]}\">21</td>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[31]}\">31</td>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[41]}\">41</td>\\n";
print "<td width=\"20\\%\" align=\"center\" bgcolor=\"${color[51]}\">51</td>\\n";
print "</tr>\\n";
print "</table>\\n";
print "<br>${status_text}\\n";
print "<br><table align=\"center\"><tr><td><form method=\"POST\" action=\"./main.cgi\">\\n";
print "<br>Please fill X and Y position in format X,Y<br>\\n";
print "<br><table align=\"center\"><tr><td><input type=\"text\"
name=\"xyposition\"></td></tr></table><br>\\n";
print "<br><table align=\"center\"><tr><td><input type=\"submit\" value=\"Start move\"
name=\"moveb\"></td></tr></table>\\n";
print "<input type=\"reset\" value=\"Clear\" name=\"resetb\"></td></tr></table>\\n";
print "</td></tr></table></form>\\n";
print "</body>\\n";
print "</html>\\n";

```

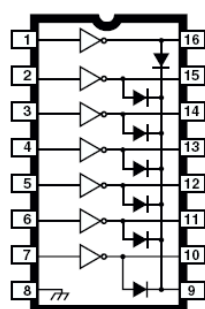
ภาคผนวก ค

วงจรเซ็นเซอร์ตรวจจับเส้นของบริษัท ETT



ภาคผนวก ง

คุณสมบัติของไอซี ULN2003

2003 THRU
2024Data Sheet
29304F**HIGH-VOLTAGE, HIGH-CURRENT
DARLINGTON ARRAYS**

Dwg. No. A-9594

Note that the ULN20xxA series (dual in-line package) and ULN20xxL series (small-outline IC package) are electrically identical and share a common terminal number assignment.

ABSOLUTE MAXIMUM RATINGS

Output Voltage, V_{CE}	
(ULN200xA and ULN200xL)	50 V
(ULN202xA and ULN202xL)	95 V
Input Voltage, V_{IN}	30 V
Continuous Output Current,	
I_C	500 mA
Continuous Input Current, I_{BI}	25 mA
Power Dissipation, P_D	
(one Darlington pair)	1.0 W
(total package)	See Graph
Operating Temperature Range,	
T_A	-20°C to +85°C
Storage Temperature Range,	
T_S	-55°C to +150°C

Ideally suited for interfacing between low-level logic circuitry and multiple peripheral power loads, the Series ULN20xxA/L high-voltage, high-current Darlington arrays feature continuous load current ratings to 500 mA for each of the seven drivers. At an appropriate duty cycle depending on ambient temperature and number of drivers turned ON simultaneously, typical power loads totaling over 230 W (350 mA x 7, 95 V) can be controlled. Typical loads include relays, solenoids, stepping motors, magnetic print hammers, multiplexed LED and incandescent displays, and heaters. All devices feature open-collector outputs with integral clamp diodes.

The ULN2003A/L and ULN2023A/L have series input resistors selected for operation directly with 5 V TTL or CMOS. These devices will handle numerous interface needs — particularly those beyond the capabilities of standard logic buffers.

The ULN2004A/L and ULN2024A/L have series input resistors for operation directly from 6 to 15 V CMOS or PMOS logic outputs.

The ULN2003A/L and ULN2004A/L are the standard Darlington arrays. The outputs are capable of sinking 500 mA and will withstand at least 50 V in the OFF state. Outputs may be paralleled for higher load current capability. The ULN2023A/L and ULN2024A/L will withstand 95 V in the OFF state.

These Darlington arrays are furnished in 16-pin dual in-line plastic packages (suffix "A") and 16-lead surface-mountable SOICs (suffix "L"). All devices are pinned with outputs opposite inputs to facilitate ease of circuit board layout. All devices are rated for operation over the temperature range of -20°C to +85°C. Most (see matrix, next page) are also available for operation to -40°C; to order, change the prefix from "ULN" to "ULQ".

FEATURES

- TTL, DTL, PMOS, or CMOS-Compatible Inputs
- Output Current to 500 mA
- Output Voltage to 95 V
- Transient-Protected Outputs
- Dual In-Line Plastic Package or Small-Outline IC Package

x = digit to identify specific device. Characteristic shown applies to family of devices with remaining digits as shown. See matrix on next page.



2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

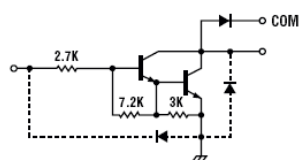
DEVICE PART NUMBER DESIGNATION

$V_{CE(MAX)}$	50 V	95 V
$I_{C(MAX)}$	500 mA	500 mA
Logic	Part Number	
5V TTL, CMOS	ULN2003A* ULN2003L*	ULN2023A* ULN2023L
6-15 V CMOS, PMOS	ULN2004A* ULN2004L*	ULN2024A ULN2024L

* Also available for operation between -40°C and +85°C. To order, change prefix from "ULN" to "ULQ".

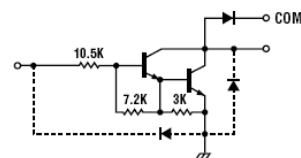
PARTIAL SCHEMATICS

ULN20x3A/L (Each Driver)

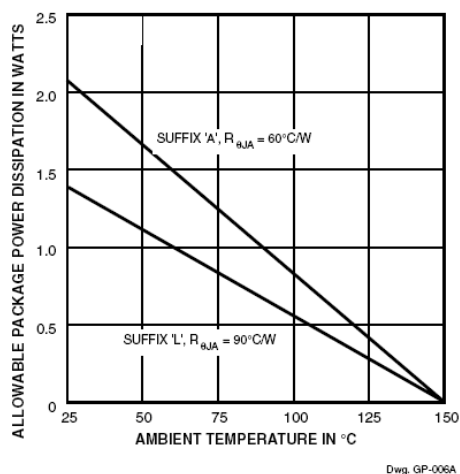


Dwg. No. A-9651

ULN20x4A/L (Each Driver)



Dwg. No. A-9698A



Dwg. GP-006A

X = Digit to identify specific device. Specification shown applies to family of devices with remaining digits as shown. See matrix above.



115 Northeast Cutoff, Box 15036
 Worcester, Massachusetts 01615-0036 (508) 853-5000
 Copyright © 1974, 1998 Allegro MicroSystems, Inc.

2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

Types ULN2003A, ULN2003L, ULN2004A, and ULN2004L
 ELECTRICAL CHARACTERISTICS at +25°C (unless otherwise noted).

Characteristic	Symbol	Test Fig.	Applicable Devices	Test Conditions	Limits			
					Min.	Typ.	Max.	Units
Output Leakage Current	I_{CEX}	1A	All	$V_{CE} = 50\text{ V}, T_A = 25^\circ\text{C}$	—	< 1	50	μA
				$V_{CE} = 50\text{ V}, T_A = 70^\circ\text{C}$	—	< 1	100	μA
		1B	ULN2004A/L	$V_{CE} = 50\text{ V}, T_A = 70^\circ\text{C}, V_{IN} = 1.0\text{ V}$	—	< 5	500	μA
Collector-Emitter Saturation Voltage	$V_{CE(SAT)}$	2	All	$I_C = 100\text{ mA}, I_B = 250\text{ }\mu\text{A}$	—	0.9	1.1	V
				$I_C = 200\text{ mA}, I_B = 350\text{ }\mu\text{A}$	—	1.1	1.3	V
				$I_C = 350\text{ mA}, I_B = 500\text{ }\mu\text{A}$	—	1.3	1.6	V
Input Current	$I_{IN(ON)}$	3	ULN2003A/L	$V_{IN} = 3.85\text{ V}$	—	0.93	1.35	mA
			ULN2004A/L	$V_{IN} = 5.0\text{ V}$	—	0.35	0.5	mA
			ULN2004A/L	$V_{IN} = 12\text{ V}$	—	1.0	1.45	mA
	$I_{IN(OFF)}$	4	All	$I_C = 500\text{ }\mu\text{A}, T_A = 70^\circ\text{C}$	50	65	—	μA
Input Voltage	$V_{IN(ON)}$	5	ULN2003A/L	$V_{CE} = 2.0\text{ V}, I_C = 200\text{ mA}$	—	—	2.4	V
				$V_{CE} = 2.0\text{ V}, I_C = 250\text{ mA}$	—	—	2.7	V
				$V_{CE} = 2.0\text{ V}, I_C = 300\text{ mA}$	—	—	3.0	V
			ULN2004A/L	$V_{CE} = 2.0\text{ V}, I_C = 125\text{ mA}$	—	—	5.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 200\text{ mA}$	—	—	6.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 275\text{ mA}$	—	—	7.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 350\text{ mA}$	—	—	8.0	V
Input Capacitance	C_{IN}	—	All		—	15	25	pF
Turn-On Delay	t_{PLH}	8	All	0.5 E_{IN} to 0.5 E_{OUT}	—	0.25	1.0	μs
Turn-Off Delay	t_{PHL}	8	All	0.5 E_{IN} to 0.5 E_{OUT}	—	0.25	1.0	μs
Clamp Diode Leakage Current	I_R	6	All	$V_R = 50\text{ V}, T_A = 25^\circ\text{C}$	—	—	50	μA
				$V_R = 50\text{ V}, T_A = 70^\circ\text{C}$	—	—	100	μA
Clamp Diode Forward Voltage	V_F	7	All	$I_F = 350\text{ mA}$	—	1.7	2.0	V

Complete part number includes suffix to identify package style: A = DIP, L = SOIC.

2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

Types ULN2023A, ULN2023L, ULN2024A, and ULN2024L
 ELECTRICAL CHARACTERISTICS at +25°C (unless otherwise noted).

Characteristic	Symbol	Test Fig.	Applicable Devices	Test Conditions	Limits			
					Min.	Typ.	Max.	Units
Output Leakage Current	I_{CEX}	1A	All	$V_{CE} = 95\text{ V}, T_A = 25^\circ\text{C}$	—	< 1	50	μA
				$V_{CE} = 95\text{ V}, T_A = 70^\circ\text{C}$	—	< 1	100	μA
		1B	ULN2024A/L	$V_{CE} = 95\text{ V}, T_A = 70^\circ\text{C}, V_{IN} = 1.0\text{ V}$	—	< 5	500	μA
Collector-Emitter Saturation Voltage	$V_{CE(SAT)}$	2	All	$I_C = 100\text{ mA}, I_B = 250\text{ }\mu\text{A}$	—	0.9	1.1	V
				$I_C = 200\text{ mA}, I_B = 350\text{ }\mu\text{A}$	—	1.1	1.3	V
				$I_C = 350\text{ mA}, I_B = 500\text{ }\mu\text{A}$	—	1.3	1.6	V
Input Current	$I_{IN(ON)}$	3	ULN2023A/L	$V_{IN} = 3.85\text{ V}$	—	0.93	1.35	mA
			ULN2024A/L	$V_{IN} = 5.0\text{ V}$	—	0.35	0.5	mA
				$V_{IN} = 12\text{ V}$	—	1.0	1.45	mA
	$I_{IN(OFF)}$	4	All	$I_C = 500\text{ }\mu\text{A}, T_A = 70^\circ\text{C}$	50	65	—	μA
Input Voltage	$V_{IN(ON)}$	5	ULN2023A/L	$V_{CE} = 2.0\text{ V}, I_C = 200\text{ mA}$	—	—	2.4	V
				$V_{CE} = 2.0\text{ V}, I_C = 250\text{ mA}$	—	—	2.7	V
				$V_{CE} = 2.0\text{ V}, I_C = 300\text{ mA}$	—	—	3.0	V
			ULN2024A/L	$V_{CE} = 2.0\text{ V}, I_C = 125\text{ mA}$	—	—	5.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 200\text{ mA}$	—	—	6.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 275\text{ mA}$	—	—	7.0	V
				$V_{CE} = 2.0\text{ V}, I_C = 350\text{ mA}$	—	—	8.0	V
Input Capacitance	C_{IN}	—	All		—	15	25	pF
Turn-On Delay	t_{PLH}	8	All	0.5 E_{IN} to 0.5 E_{OUT}	—	0.25	1.0	μs
Turn-Off Delay	t_{PHL}	8	All	0.5 E_{IN} to 0.5 E_{OUT}	—	0.25	1.0	μs
Clamp Diode Leakage Current	I_R	6	All	$V_R = 95\text{ V}, T_A = 25^\circ\text{C}$	—	—	50	μA
				$V_R = 95\text{ V}, T_A = 70^\circ\text{C}$	—	—	100	μA
Clamp Diode Forward Voltage	V_F	7	All	$I_F = 350\text{ mA}$	—	1.7	2.0	V

Complete part number includes suffix to identify package style: A = DIP, L = SOIC.



115 Northeast Cutoff, Box 15036
 Worcester, Massachusetts 01615-0036 (508) 853-5000

2003 THRU 2024
**HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS**

TEST FIGURES

FIGURE 1A

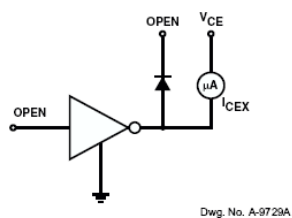


FIGURE 1B

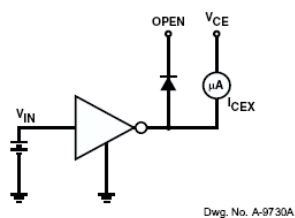


FIGURE 2

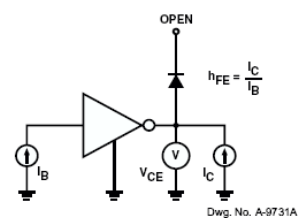


FIGURE 3

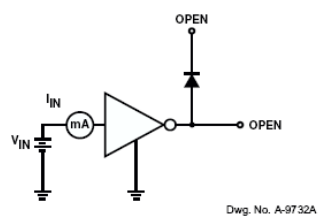


FIGURE 4

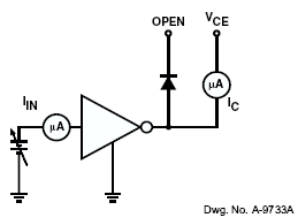


FIGURE 5

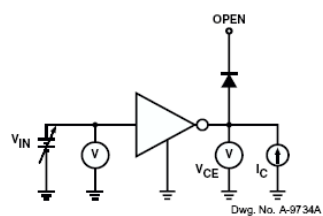


FIGURE 6

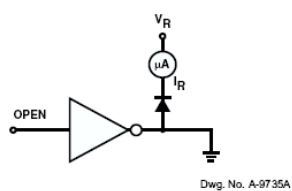


FIGURE 7

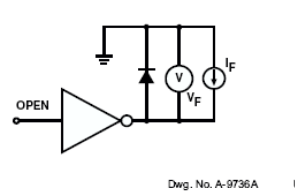
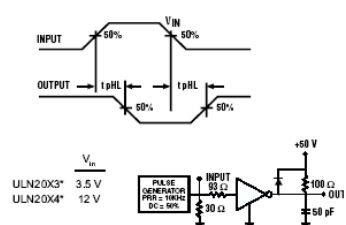


FIGURE 8

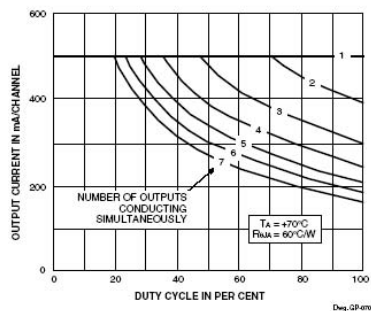


* Complete part number includes a final letter to indicate package.

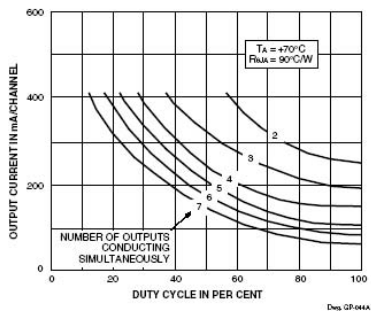
X = Digit to identify specific device. Specification shown applies to family of devices with remaining digits as shown.

2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

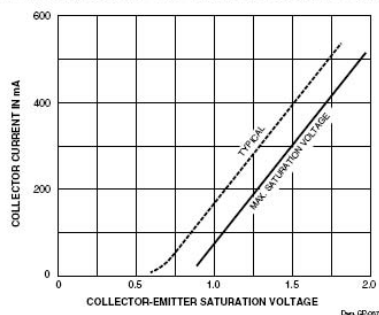
**ALLOWABLE COLLECTOR CURRENT
AS A FUNCTION OF DUTY CYCLE**
(Dual In-line-Packaged Devices, Suffix 'A')



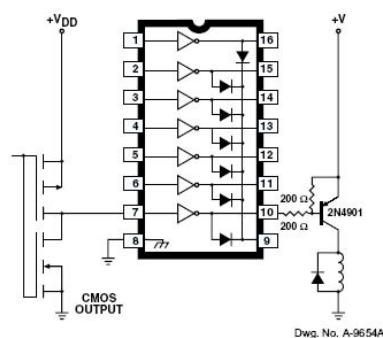
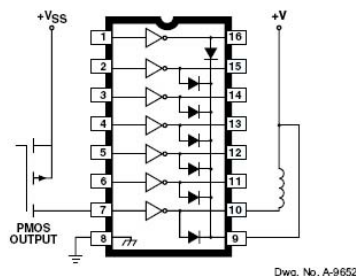
(Small-Outline-Packaged Devices, Suffix 'L')



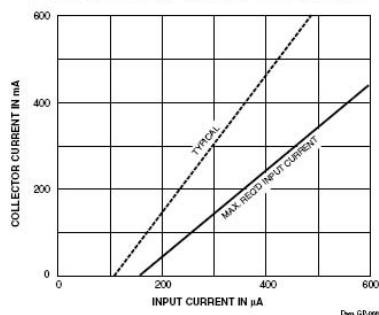
**SATURATION VOLTAGE
AS A FUNCTION OF COLLECTOR CURRENT**



TYPICAL APPLICATIONS



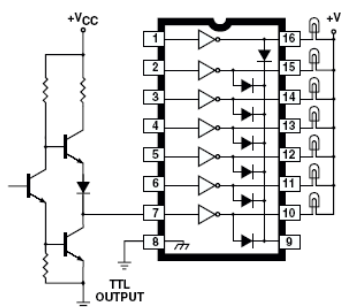
**COLLECTOR CURRENT AS A
FUNCTION OF INPUT CURRENT**



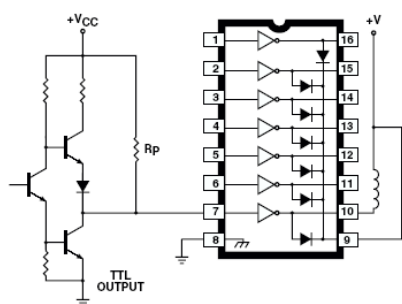
115 Northeast Cutoff, Box 15036
Worcester, Massachusetts 01615-0036 (508) 853-5000

2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

TYPICAL APPLICATIONS



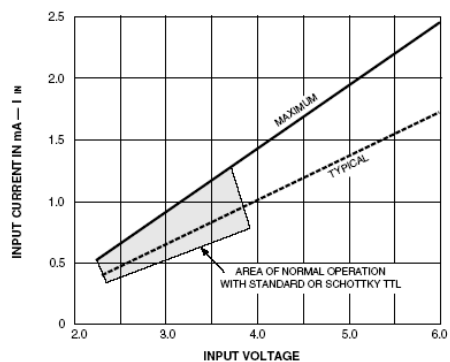
Dwg. No. A-9653A



Dwg. No. A-10,175

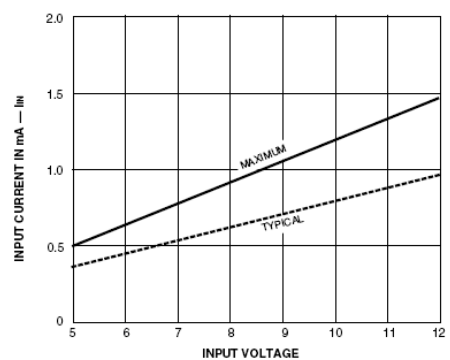
INPUT CURRENT
AS A FUNCTION OF INPUT VOLTAGE

Types ULN2003A, ULN2003L, ULN2023A, and
 ULN2023L



Dwg. GP-660

Types ULN2004A, ULN2004L, ULN2024A, and
 ULN2024L



Dwg. GP-660-1

Dimensions in Inches
(controlling dimensions)



Dimension in Millimeters
(for reference only)



NOTES: 1. Leads 1, 8, 9, and 16 may be half leads at vendor's option.
2. Lead thickness is measured at seating plane or below.
3. Lead spacing tolerance is non-cumulative.
4. Exact body and lead configuration at vendor's option within limits shown.

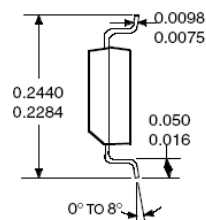
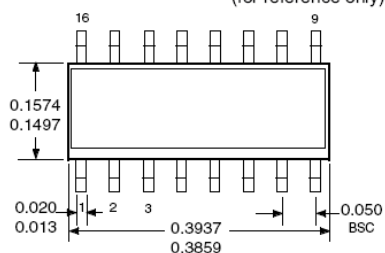


115 Northeast Cutoff, Box 15036
Worcester, Massachusetts 01615-0036 (508) 853-5000

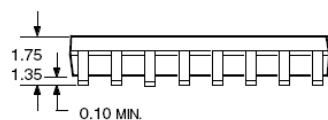
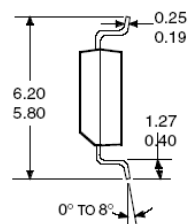
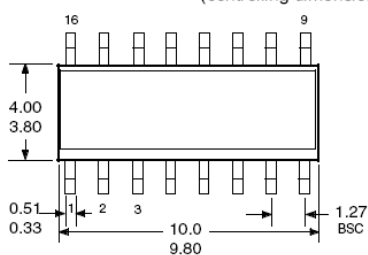
2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

PACKAGE DESIGNATOR "L"

Dimensions in Inches
(for reference only)



Dimension in Millimeters
(controlling dimensions)



Dwg. MA-007-16A mm

- NOTES: 1. Lead spacing tolerance is non-cumulative.
2. Exact body and lead configuration at vendor's option within limits shown.

2003 THRU 2024
HIGH-VOLTAGE,
HIGH-CURRENT
DARLINGTON ARRAYS

The products described here are manufactured under one or more U.S. patents or U.S. patents pending.

Allegro MicroSystems, Inc. reserves the right to make, from time to time, such departures from the detail specifications as may be required to permit improvements in the performance, reliability, or manufacturability of its products. Before placing an order, the user is cautioned to verify that the information being relied upon is current.

Allegro products are not authorized for use as critical components in life-support devices or systems without express written approval.

The information included herein is believed to be accurate and reliable. However, Allegro MicroSystems, Inc. assumes no responsibility for its use; nor for any infringement of patents or other rights of third parties which may result from its use.

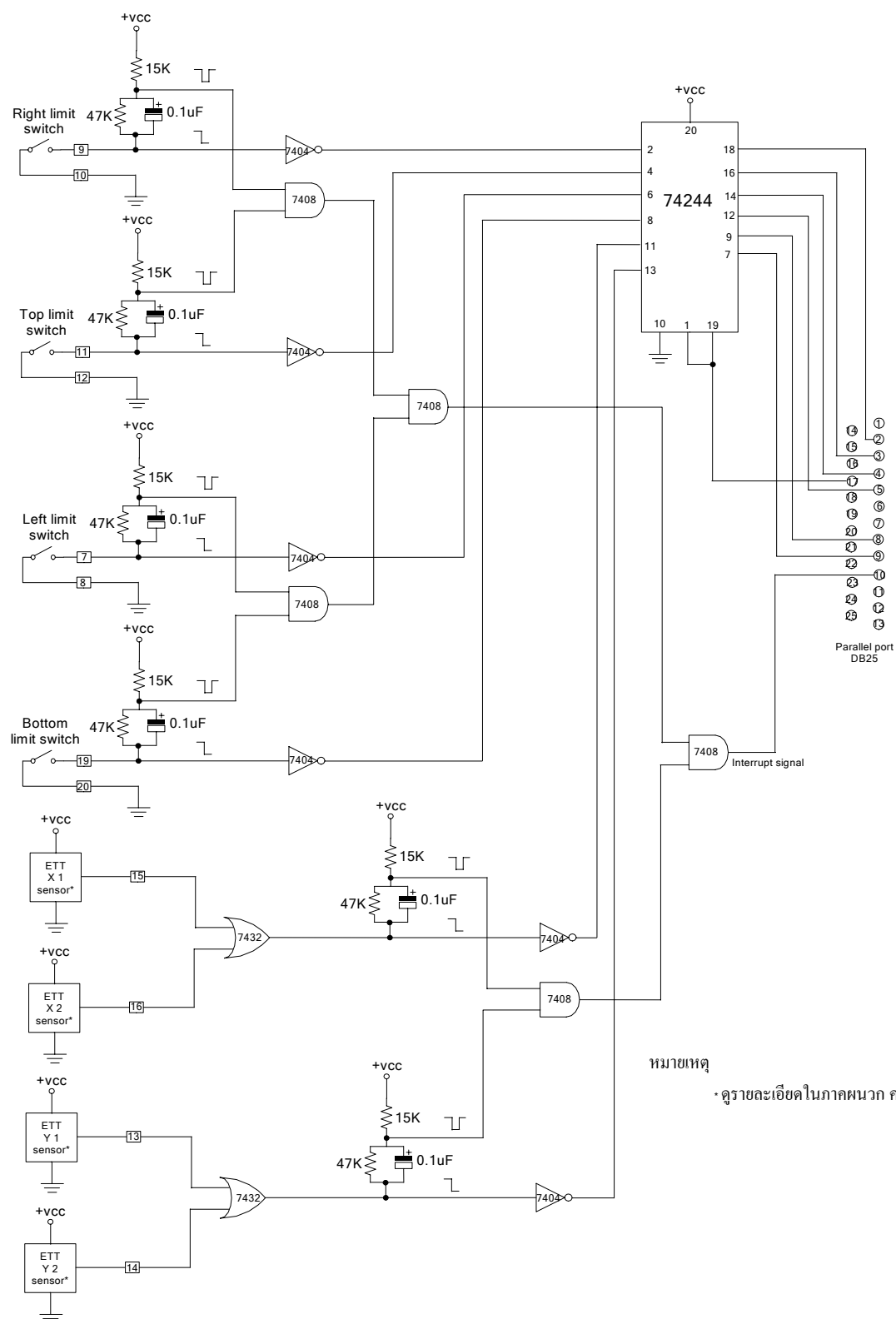


115 Northeast Cutoff, Box 15036
Worcester, Massachusetts 01615-0036 (508) 853-5000

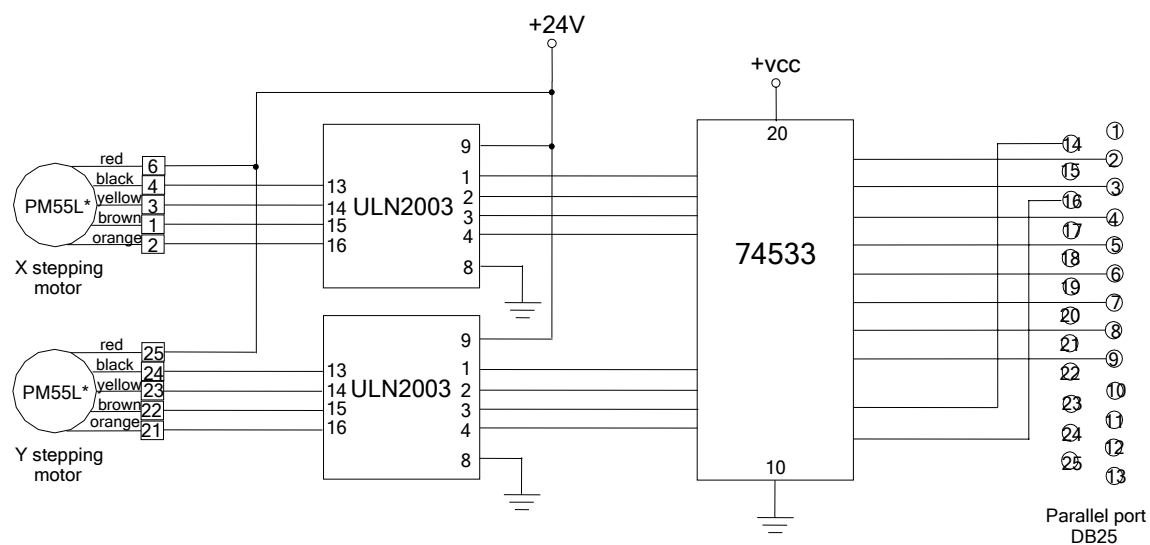
ภาคผนวก จ

วงจรที่สมบูรณ์ของ Crane interface card

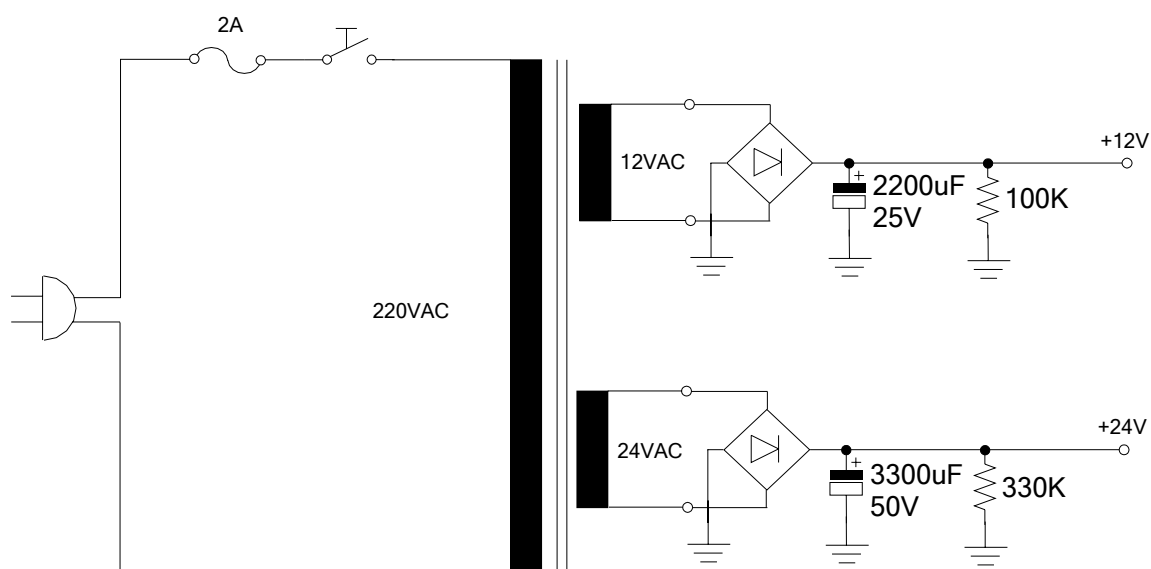
1. วงจรส่วนติดต่อกับเครน



2. วงจรขับสเต็ปมอเตอร์

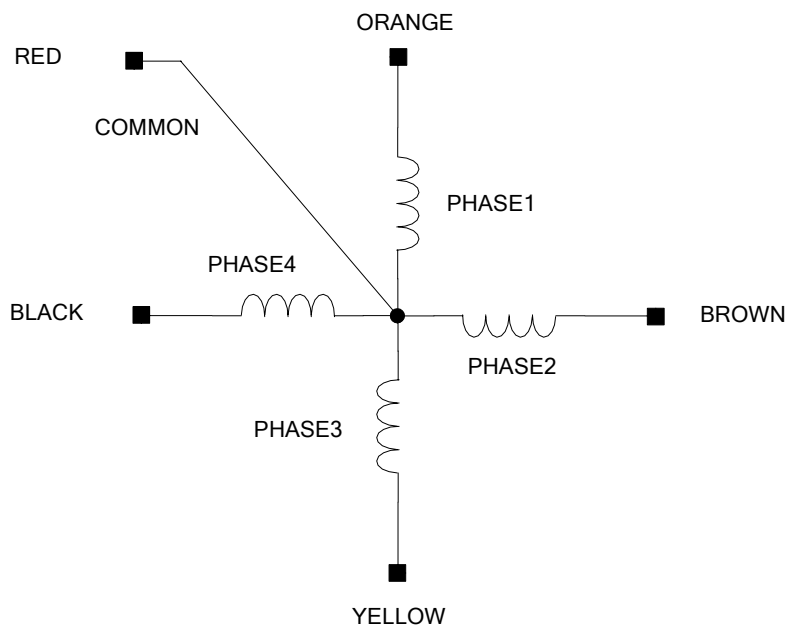


3. วงจรจ่ายแรงดัน



ภาคผนวก จ

คุณสมบัติของสเต็ปป์มอเตอร์ PM55L



STEPPING	7.5 DEGREE:STEP
RESISTANCE	200 OHM
PHASE INDUCTANCE	78mH

บรรณานุกรม

- [1] Craig Hollabaugh Ph.D. (2002) : “Embedded Linux ® : Hareware, Software , and Interfacing” , 7 2002
- [2] John Lambardo. (2002) : “Embedded Linux” , New Riders Publishing , 5 2002
- [3] Daniel P. Bovet and Marco Cesati. (2000) : “Understanding the Linux Kernel” , O'Reilly & Associates , September 2000
- [4] Michael Barr. (1999) : “Programming Embedded Systems in C and C++” , O'Reilly & Associates , 1 1999
- [5] Alessandro Rubini and Jonathan Corbet. (2001) : “Linux Device Drivers, 2nd Edition” , O'Reilly & Associates , 6 2001
- [6] Karim Yaghmour. (2003) : “Building Embedded Linux Systems” , O'Reilly & Associates , 4 2003
- [7] John Catsoulis. (2002) : “Designing Embedded Hardware” , O'Reilly & Associates , 11 2002
- [8] Krut Wall , Mark Warkson and Mark Whitis. (1999) : “Linux Programming” , Sams , 1999
- [9] Mark Mitchell , Jeffrey Oldham and Alex Samuel. (2001) : “Advance Linux Programming” , New Riders Publishing , 2001